

Security Audit

STBL (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Code Quality	9
Audit Resources	9
Dependencies	9
Severity Definitions	10
Status Definitions	11
Audit Findings	12
Centralisation	99
Conclusion	100
Our Methodology	101
Disclaimers	103
About Hashlock	104

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The STBL team partnered with Hashlock to conduct a security audit of their code. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

STBL is a decentralized stablecoin infrastructure protocol introducing "Stablecoin 2.0" through a Money-as-a-Service (MaaS) architecture, enabling ecosystems, institutions, and enterprises to launch programmable, ecosystem-specific stablecoins backed by tokenized real-world assets (RWAs). The ecosystem is built around a tri-token model consisting of USST, an over-collateralized stablecoin designed for liquidity and interoperability; YLD, a yield-bearing NFT representing rights to collateral yield; and STBL, the governance and value-accrual token responsible for protocol incentives and coordination. Unlike traditional stablecoin systems where issuers retain all yield, STBL introduces a "Principal-Yield Split" mechanism that separates the stable asset from its yield stream, improving transparency, capital efficiency, and monetary sovereignty while maintaining on-chain auditability and cross-chain interoperability. The protocol leverages tokenized RWAs, including U.S. Treasury-backed assets and private credit products, and supports white-labeled Ecosystem Specific Stablecoins (ESS), allowing partners to customize monetary policies, incentives, and yield distribution while remaining connected through the USST liquidity layer.

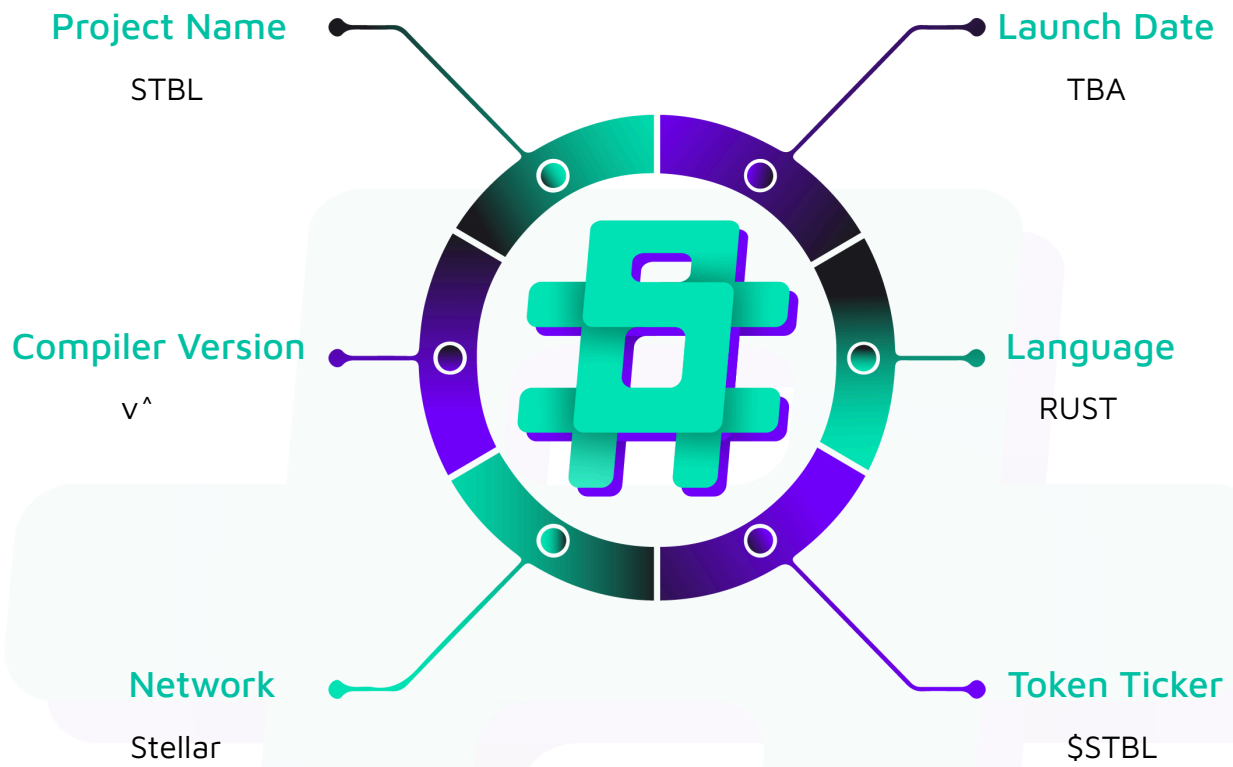
Project Name: STBL

Project Type: DeFi

Website: <https://www.stbl.com/>

Logo:



Visualised Context:

Project Visuals:



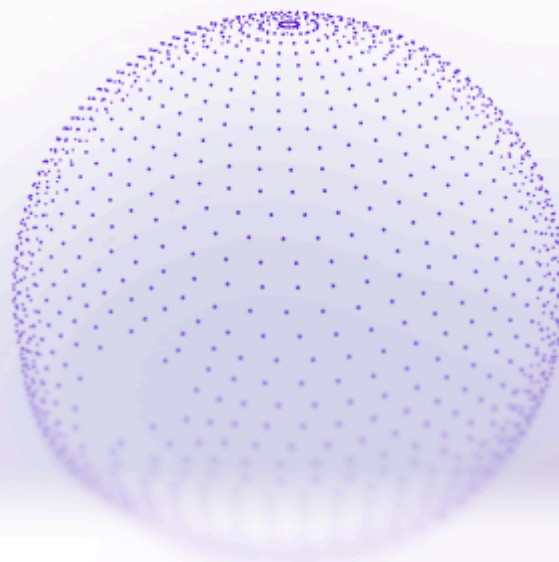
[Rewards](#) [Blog](#) [About](#) [Docs](#) [Get \\$STBL](#)

[Access App](#)

OWN YOUR ECONOMY

Empowering ecosystems globally to issue their own money & manage yield.

Every ecosystem is unique, its currency should be too. With Stablecoin 2.0, this becomes reality.



STBL's Money-as-a-Service (MaaS) infrastructure

Launch your own programmable, yield-sharing stablecoin. Anchored to USST, the universal, over-collateralized stablecoin backed by high-quality RWAs providing stability and global liquidity.



Audit Scope

We at Hashlock audited the Rust code within the STBL project, the scope of work included a comprehensive review of the programs and modules listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	STBL Programs
Network	Stellar
Language	Rust
Audit Date	May, 2026
GitHub Repository	https://github.com/USD-Pi-Protocol/stellar-core-contracts
Program 1	contracts/*
Program 2	libraries/*
Audited GitHub Commit Hash	54070e56a5789c93373984718925f28363faed34

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Programs Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

2 High severity vulnerabilities

7 Medium severity vulnerabilities

15 Low severity vulnerabilities

8 QAs

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Code Quality

This audit scope involves the smart contracts of the STBL project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the STBL project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-severity issues should be resolved before deployment. While they do not typically lead to a complete loss of funds, they may result in partial loss of funds or unintended behavior under certain conditions.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] AirdropAssetIssuer#withdraw - Fee reserves can let expiring positions avoid confiscation

Description

The airdrop issuer's withdrawal solvency check values the issuer contract's full token balance, including tokens already accounted as treasury fee reserves. As a result, a DLT holder can withdraw before expiry even when the collateral balance excluding fees is insufficient, avoiding the later `withdraw_expired()` path that would confiscate the expired position's collateral to the treasury.

Vulnerability Details

`deposit_erc20()` records deposit and insurance fees in vault-wide fee buckets while also tracking user liabilities in `deposit_value_usd` in `contracts/airdrop-asset-issuer/src/lib.rs:226-236`. Airdrop yield distribution similarly records protocol yield fees in `vault.yield_fees` in `contracts/airdrop-asset-issuer/src/lib.rs:555-584`, and `withdraw_fees()` later expects treasury to be able to withdraw the sum of all fee buckets in `contracts/airdrop-asset-issuer/src/lib.rs:613-647`.

The contract already recognizes that fee reserves should be excluded from collateral accounting in `calculate_price_differentiation_internal()`, which subtracts accumulated fees from the live token balance before valuing distributable yield in `contracts/airdrop-asset-issuer/src/lib.rs:180-190`.

However, `withdraw_erc20()` performs its solvency check against `current_balance` directly in `contracts/airdrop-asset-issuer/src/lib.rs:267-274`. It does not subtract `deposit_fees`, `withdraw_fees`, `insurance_fees`, or `yield_fees` before checking whether the vault value covers `vault.deposit_value_usd`. If the fee-excluded collateral value is below outstanding user liabilities but the full balance, including fee reserves, is still above them, the withdrawal proceeds and transfers tokens to the user in

#hashlock.

`contracts/airdrop-asset-issuer/src/lib.rs:286-303.`

For DLT assets, this can bypass the intended expiry outcome. User withdrawals are blocked only after the configured duration elapses in `contracts/airdrop-asset-issuer/src/lib.rs:401-410`, while `withdraw_expired()` lets treasury finalize an expired DLT position after that duration in `contracts/airdrop-asset-issuer/src/lib.rs:751-794`. During the pre-expiry window where fee-excluded collateral is insufficient but fee-inclusive collateral still passes the current check, a holder can withdraw at par and avoid the later treasury confiscation path. The shortfall is externalized to global fee reserves, remaining users, or treasury fee collection.

The exploit window is bounded but practical: the global collateral shortfall must be no larger than the global fee reserves. Normal fee accrual widens this window, and the condition can arise from price movement, rounding, prior withdrawals, or airdrop balance changes before a DLT position expires.

Proof of Concept

The first test demonstrates the higher-impact DLT bypass variant. Alice and Bob have equivalent DLT positions. After a price drop, the issuer is insolvent if fee reserves are excluded, but appears solvent because `withdraw_erc20()` values the full token balance. Alice withdraws before expiry using those fee reserves as hidden backing; Bob does not withdraw, and his equivalent position is later confiscated by treasury through `withdraw_expired()`.

Insert this test into `integration-tests/tests/test_dpt_calls.rs`. It uses the imports already present in that file, including `AssetIssuer`, `AssetIssuerClient`, `TestIntegratedSystem`, `Ledger`, `LedgerInfo`, `vec`, `Bytes`, `String`, `Vec`, `VaultClient`, `ExternalOracleClient`, `StblTokenClient`, `UsstTokenClient`, and `YldTokenClient`.

```
#[test]
fn test_dlt_holder_escapes_expiry_confiscation_using_fee_reserves() {
    let system = TestIntegratedSystem::deploy();
    let env = &system.env;
    let register_client = register::StblRegisterClient::new(env, &system.registry);
    let benji_decimals: u32 = 6;
    let benji_token = env.register(
```

```

        usst_token::UsstToken,
        (system.admin.clone(), Some(benji_decimals)),
    );
let benji_token_client = StblTokenClient::new(env, &benji_token);
system.grant_minting_access(&benji_token, &system.admin);
let asset_name = String::from_str(env, "BenJi");
let description = String::from_str(env, "Franklin OnChain U.S. Government Money
Fund");
let asset_id =
    register_client.add_asset(&system.admin, &asset_name, &description, &1, &3,
&true);
let rwa_oracle = env.register(rwa_oracle::RwaOracle, (system.admin.clone(),));
let rwa_oracle_client = ExternalOracleClient::new(env, &rwa_oracle);
rwa_oracle_client.set_price_data(&1_000_000_000);
let oracle = env.register(
    asset_oracle::AssetOracle,
    (system.admin.clone(), rwa_oracle.clone(), &9u32, &2_592_000u64),
);
let issuer = env.register(
    AssetIssuer,
    (system.admin.clone(), system.registry.clone(), asset_id),
);
let distributor = env.register(
    asset_yield_distributor::AssetYieldDistributor,
    (system.admin.clone(), system.registry.clone(), asset_id),
);
let deposit_fee_10_percent = 100_000_000u128;
let duration = 86_400u64;
let fees: Vec<u128> = vec![env, deposit_fee_10_percent, 0, 0, 0];
register_client.setup_asset(
    &system.admin,
    &asset_id,
    &vec![
        env,
        benji_token.clone(),
        issuer.clone(),
        distributor.clone(),
        oracle.clone(),
    ],
);

```

```

    ],
    &0,
    &u128::pow(10, 24),
    &fees,
    &duration,
    &0,
    &Bytes::new(env),
);
let asset_issuer = AssetIssuerClient::new(env, &issuer);
let asset_vault_client = VaultClient::new(env, &issuer);
let yld = YldTokenClient::new(env, &system.yld);
let usst = UsstTokenClient::new(env, &system.usst);
let deposit_amount = 1_000u128 * u128::pow(10, benji_decimals);
system.mint_tokens_to_user(&benji_token, &system.alice, Some(deposit_amount));
system.mint_tokens_to_user(&benji_token, &system.bob, Some(deposit_amount));
benji_token_client.approve(
    &system.alice,
    &issuer,
    &(deposit_amount as i128),
    &(env.ledger().sequence() + 100),
);
benji_token_client.approve(
    &system.bob,
    &issuer,
    &(deposit_amount as i128),
    &(env.ledger().sequence() + 100),
);
let alice_token_id = asset_issuer.deposit(&system.alice, &deposit_amount, &None);
let bob_token_id = asset_issuer.deposit(&system.bob, &deposit_amount, &None);
let vault_after_deposits = asset_vault_client.fetch_vault_data();
assert_eq!(
    vault_after_deposits.deposit_fees,
    200u128 * u128::pow(10, 18)
);
assert_eq!(
    vault_after_deposits.deposit_value_usd,
    1_800u128 * u128::pow(10, 18)
);

```

```

env.ledger().set(LedgerInfo {
  sequence_number: env.ledger().sequence() + 1,
  timestamp: env.ledger().timestamp() + 5,
  ..env.ledger().get()
});
rwa_oracle_client.set_price_data(&950_000_000);
// Correct fee-excluded solvency would fail:
// collateral excluding the 200 fee tokens is worth 1,710, below 1,800
liabilities.
let fee_excluded_balance_value = 1_710u128 * u128::pow(10, 18);
let full_balance_value = 1_900u128 * u128::pow(10, 18);
assert!(fee_excluded_balance_value < vault_after_deposits.deposit_value_usd);
assert!(full_balance_value >= vault_after_deposits.deposit_value_usd);
// Current code lets Alice withdraw before expiry by counting fee reserves.
yld.approve(
  &system.alice,
  &system.core,
  &alice_token_id,
  &(env.ledger().sequence() + 100),
);
usst.approve(
  &system.alice,
  &system.core,
  &i128::MAX,
  &(env.ledger().sequence() + 100),
);
let alice_asset_before = benji_token_client.balance(&system.alice);
asset_issuer.withdraw(&system.alice, &alice_token_id, &None);
assert!(benji_token_client.balance(&system.alice) > alice_asset_before);
// Bob's equivalent unwithdrawn position is left to expire and is confiscated by
treasury.
env.ledger().set(LedgerInfo {
  sequence_number: env.ledger().sequence() + ((duration + 1) / 5) as u32 + 1,
  timestamp: env.ledger().timestamp() + duration + 1,
  ..env.ledger().get()
});
rwa_oracle_client.set_price_data(&950_000_000);
let treasury_before = benji_token_client.balance(&system.treasury);

```

```

    asset_issuer.withdraw_expired(&system.treasury, &bob_token_id);
    assert!(benji_token_client.balance(&system.treasury) > treasury_before);
    // Fee accounting still expects 200 fee tokens, but the shared balance was
    consumed.
    assert!(asset_issuer.try_withdraw_fees(&system.treasury).is_err());
}

```

The second test demonstrates the narrower accounting window where the airdrop issuer is insolvent if fee reserves are excluded, but appears solvent because `withdraw_erc20()` values the full token balance.

```

#[test]
fn test_dpt_withdraw_can_consume_fee_reserves_as_collateral() {
    let system = TestIntegratedSystem::deploy();
    let env = &system.env;
    let register_client = register::StblRegisterClient::new(env, &system.registry);
    let benji_decimals: u32 = 6;
    let benji_token = env.register(
        usst_token::UsstToken,
        (system.admin.clone(), Some(benji_decimals)),
    );
    let benji_token_client = StblTokenClient::new(env, &benji_token);
    system.grant_minting_access(&benji_token, &system.admin);
    let asset_name = String::from_str(env, "BenJi");
    let description = String::from_str(env, "Franklin OnChain U.S. Government Money Fund");
    let asset_id =
        register_client.add_asset(&system.admin, &asset_name, &description, &1, &4,
    &true);
    let rwa_oracle = env.register(rwa_oracle::RwaOracle, (system.admin.clone(),));
    let rwa_oracle_client = ExternalOracleClient::new(env, &rwa_oracle);
    rwa_oracle_client.set_price_data(&1_000_000_000);
    let oracle = env.register(
        asset_oracle::AssetOracle,
        (system.admin.clone(), rwa_oracle.clone(), &9u32, &2_592_000u64),
    );
    let issuer = env.register(

```

```

    AssetIssuer,
    (system.admin.clone(), system.registry.clone(), asset_id),
);
let distributor = env.register(
    asset_yield_distributor::AssetYieldDistributor,
    (system.admin.clone(), system.registry.clone(), asset_id),
);
let deposit_fee_10_percent = 100_000_000u128;
let fees: Vec<u128> = vec![env, deposit_fee_10_percent, 0, 0, 0];
register_client.setup_asset(
    &system.admin,
    &asset_id,
    &vec![
        env,
        benji_token.clone(),
        issuer.clone(),
        distributor.clone(),
        oracle.clone(),
    ],
    &0,
    &u128::pow(10, 24),
    &fees,
    &86_400,
    &0,
    &Bytes::new(env),
);
let asset_issuer = AssetIssuerClient::new(env, &issuer);
let asset_vault_client = VaultClient::new(env, &issuer);
let yld = YldTokenClient::new(env, &system.yld);
let usst = UsstTokenClient::new(env, &system.usst);
let deposit_amount = 1_000u128 * u128::pow(10, benji_decimals);
system.mint_tokens_to_user(&benji_token, &system.alice, Some(deposit_amount));
benji_token_client.approve(
    &system.alice,
    &issuer,
    &(deposit_amount as i128),
    &(env.ledger().sequence() + 100),
);

```

```

let token_id = asset_issuer.deposit(&system.alice, &deposit_amount, &None);
let vault_after_deposit = asset_vault_client.fetch_vault_data();
let fee_reserve_raw = 100u128 * u128::pow(10, benji_decimals);
assert_eq!(vault_after_deposit.deposit_fees, 100u128 * u128::pow(10, 18));
assert_eq!(vault_after_deposit.deposit_value_usd, 900u128 * u128::pow(10, 18));
env.ledger().set(LedgerInfo {
    sequence_number: env.ledger().sequence() + 1,
    timestamp: env.ledger().timestamp() + 5,
    ..env.ledger().get()
});
rwa_oracle_client.set_price_data(&950_000_000);
let full_balance_value = 950u128 * u128::pow(10, 18);
let fee_excluded_balance_value = 855u128 * u128::pow(10, 18);
assert!(fee_excluded_balance_value < vault_after_deposit.deposit_value_usd);
assert!(full_balance_value >= vault_after_deposit.deposit_value_usd);
yld.approve(
    &system.alice,
    &system.core,
    &token_id,
    &(env.ledger().sequence() + 100),
);
usst.approve(
    &system.alice,
    &system.core,
    &i128::MAX,
    &(env.ledger().sequence() + 100),
);
asset_issuer.withdraw(&system.alice, &token_id, &None);
let issuer_balance_after_withdraw = benji_token_client.balance(&issuer) as u128;
assert!(issuer_balance_after_withdraw < fee_reserve_raw);
assert!(asset_issuer.try_withdraw_fees(&system.treasury).is_err());
}

```

The critical condition is:

```
fee_excluded_collateral_value < vault.deposit_value_usd <= full_balance_value
```


Under corrected rules, Alice's withdrawal would revert because only fee-excluded collateral should back user liabilities. In the current implementation, the withdrawal succeeds, and the remaining balance is no longer sufficient to satisfy the recorded fee reserves.

Impact

Global fee reserves can be consumed as user collateral. The withdrawing holder does not receive more than the recorded YLD claim, but the shortfall is funded by fee buckets that should remain reserved for treasury. Those fee buckets can include fees paid by other positions, and after the withdrawal, treasury fee collection or later withdrawals can fail.

Second, for DLT positions, the same accounting bug can bypass expiry confiscation. A holder can exit an undercollateralized pool at par before expiry and avoid the later `withdraw_expired()` outcome that would apply to an equivalent expired position. In that scenario, the avoided loss can approach the value of the holder's YLD claim, not merely the fees paid by that holder.

Recommendation

Make all position-settlement solvency checks use fee-excluded collateral. Before comparing against `vault.deposit_value_usd`, compute accumulated fees, convert them into token terms, subtract them from `current_balance`, and value only the remaining collateral balance. Normal `withdraw()` should revert if fee-excluded collateral cannot cover the outstanding liability, and the post-withdraw fee-excluded balance should still cover remaining user liabilities. `withdraw_expired()` should also avoid using fee reserves as position collateral and should follow an explicit loss-settlement rule if an expired position is undercollateralized. This keeps treasury fee reserves separate from collateral backing user redemptions and prevents pre-expiry withdrawals from bypassing DLT confiscation.

Status

Resolved

[H-02] Expired YLD positions can keep earning yield

Description

Expired LT positions can remain active in the yield distributor when the YLD holder leaves claimable rewards unclaimed. The LT `withdraw_expired()` path validates expiry, attempts to move collateral to treasury, and then disables staking in `contracts/asset-issuer/src/lib.rs:727-785`; if the later staking call reverts, the whole transaction reverts and the expired staking position stays active. The holder can then keep accumulating yield from later distributions and claim it when desired. The same vulnerable sequence is duplicated for DLT assets in `contracts/airdrop-asset-issuer/src/lib.rs:751-809`.

Vulnerability Details

The issuer only attempts to claim rewards if `yld_metadata.is_disabled` is already true, which is not the normal active position state in `contracts/asset-issuer/src/lib.rs:764-778`. For an active YLD NFT, the function proceeds directly to `disable_staking()`.

The distributor materializes pending rewards before disabling staking. If the position has any nonzero earned amount, `disable_staking()` reverts with `UnclaimedRewardsExist` in `contracts/asset-yield-distributor/src/lib.rs:351-360`. A user can therefore wait for a normal yield distribution and simply avoid calling `claim()`.

The same `disable_staking()` invariant also affects ordinary user withdrawals and YLD splits: `internal_withdraw()` disables stake before `Core.exit()` in `contracts/asset-issuer/src/lib.rs:392-412`, and `split_yld()` routes through `modify_staking()` to disable the original token before enabling split tokens in `contracts/yld-splitter/src/lib.rs:281-304` and `contracts/asset-issuer/src/lib.rs:590-625`. These cases are user-recoverable by claiming first, so they are supporting evidence for the brittle staking-settlement design rather than separate high-impact issues.

Yield distribution is enough to create this state because `distribute_reward()` updates the global reward index when total stake is nonzero in `contracts/asset-yield-distributor/src/lib.rs:275-319`. Pending rewards are then calculated from the global/per-position reward index delta in

contracts/asset-yield-distributor/src/lib.rs:242-265. Because `claim()` pays rewards to the current YLD owner, the expired holder can still receive those rewards while the position remains active in contracts/asset-yield-distributor/src/lib.rs:379-404.

Proof of Concept

Insert the following test into `integration-tests/tests/test_withdraw_flow.rs`, next to the existing withdrawal-flow tests. It reuses that file's existing LT setup helper, then distributes yield, confirms pending rewards are nonzero, advances past expiry, shows that treasury cannot complete `withdraw_expired()`, accumulates more post-expiry rewards, and finally shows the expired holder can claim the accumulated yield.

```
#[test]
fn test_withdraw_expired_reverts_with_unclaimed_rewards() {
    let (system, issuer) = run_setup_till_first_deposit(1);
    let env = &system.env;
    let register_client = register::StblRegisterClient::new(env, &system.registry);
    let asset_issuer = AssetIssuerClient::new(env, &issuer);
    let asset_id: u64 = asset_issuer.fetch_asset_id();
    let asset_definition: AssetDefinition =
register_client.fetch_asset_data(&asset_id).unwrap();
    let oracle_client = AssetOracleClient::new(env, &asset_definition.oracle);
    let rwa_oracle_addr: Address = oracle_client.get_rwa_oracle();
    let rwa_oracle_client = ExternalOracleClient::new(env, &rwa_oracle_addr);
    let distributor_client = YieldDistributorClient::new(env,
&asset_definition.reward_distributor);
    let benji_token_client = StblTokenClient::new(env, &asset_definition.token);
    let issuer_access_control_client = AccessControlClient::new(env, &issuer);
    let first_deposit_id: u32 = 1;
    issuer_access_control_client.grant_role(
        &system.admin,
        &system.treasury,
        &String::from_str(env, YD_ROLE),
    );
    system.env.ledger().set(LedgerInfo {
        sequence_number: env.ledger().sequence() + (DAY / 5) as u32,
        timestamp: env.ledger().timestamp() + DAY + 1,
```

```

        ..system.env.ledger().get()
    });
    rwa_oracle_client.set_price_data(&1_100_000_000);
    asset_issuer.distribute_yield(&system.treasury);
    let pending_rewards =
distributor_client.calculate_rewards_earned(&(first_deposit_id as u64));
    assert!(pending_rewards > 0);
    system.env.ledger().set(LedgerInfo {
        sequence_number: env.ledger().sequence() + ((2 * DAY) / 5) as u32,
        timestamp: env.ledger().timestamp() + (2 * DAY) + 1,
        ..system.env.ledger().get()
    });
    rwa_oracle_client.set_price_data(&1_100_000_000);
    let treasury_balance_before = benji_token_client.balance(&system.treasury);
    assert!(asset_issuer
        .try_withdraw_expired(&system.treasury, &first_deposit_id)
        .is_err());
    assert_eq!(
        benji_token_client.balance(&system.treasury),
        treasury_balance_before
    );
    let pending_after_failed_expiry =
        distributor_client.calculate_rewards_earned(&(first_deposit_id as u64));
    for price in [1_200_000_000u128, 1_300_000_000u128, 1_400_000_000u128] {
        system.env.ledger().set(LedgerInfo {
            sequence_number: env.ledger().sequence() + (DAY / 5) as u32,
            timestamp: env.ledger().timestamp() + DAY + 1,
            ..system.env.ledger().get()
        });
        rwa_oracle_client.set_price_data(&price);
        asset_issuer.distribute_yield(&system.treasury);
    }
    let accumulated_rewards =
        distributor_client.calculate_rewards_earned(&(first_deposit_id as u64));
    assert!(accumulated_rewards > pending_after_failed_expiry);
    let alice_balance_before = benji_token_client.balance(&system.alice);
    let claimed = distributor_client.claim(&system.alice, &(first_deposit_id as
u64));

```

```

assert_eq!(claimed, accumulated_rewards);
assert!(benji_token_client.balance(&system.alice) > alice_balance_before);
}

```

The test requires `AccessControlClient`, `YieldDistributorClient`, and `YD_ROLE` imports if they are not already present in `integration-tests/tests/test_withdraw_flow.rs`.

Run it with:

```
cargo test -p integration-tests test_withdraw_expired_reverts_with_unclaimed_rewards
```

Impact

An overdue user can keep the expired YLD position active and continue receiving yield after the position should have been closed by the treasury. The collateral remains in the issuer contract, but because staking is not disabled, future yield distributions can keep increasing the expired holder's claimable rewards. The holder is incentivized not to claim early because keeping rewards unclaimed preserves the expiry blockage; instead, the holder can accumulate rewards over repeated distributions and claim them later. The leakage can grow over time, and the operator cannot reliably stop it for an already affected position without a contract change or user cooperation.

Recommendation

Add an issuer-only forced expiry settlement path in the distributor that clears, redirects, or finalizes the position's pending rewards without reverting on nonzero earned. Both LT and DLT `withdraw_expired()` implementations should call that settlement before `disable_staking()`. Operational mitigations, such as pausing yield distribution or asking users to claim, are incomplete because they either deny rewards to honest active positions or require cooperation from the expired holder.

Status

Resolved

Medium

[M-01] Outdated stellar-tokens - Expired allowances can remain spendable

Description

USST and STBL depend on `stellar-tokens = 0.5.0`, a version with a known allowance-expiry bug in the upstream fungible token implementation, as shown in `contracts/usst-token/Cargo.toml:11` and `contracts/stbl-token/Cargo.toml:11`. Because both tokens route approvals and allowance-based movement through OpenZeppelin's base token code, a spender can consume an allowance after its `live_until_ledger` has passed if the temporary allowance entry is still present.

Vulnerability Details

The public USST `allowance()` view delegates to `Base::allowance`, and `approve()` delegates to `Base::approve` in `contracts/usst-token/src/lib.rs:234-239`. `transfer_from()` also delegates the actual allowance spending to `Base::transfer_from` after only applying blacklist checks in `contracts/usst-token/src/lib.rs:218-223`. STBL is even more direct: it uses the default `FungibleToken` and `FungibleBurnable` implementations from the same vulnerable dependency in `contracts/stbl-token/src/lib.rs:129-137`.

OpenZeppelin fixed this upstream in `stellar-tokens v0.6.0` by making expired allowance data resolve to zero before it reaches `spend_allowance`. In the 0.5.0 behavior used here, the view path can report an allowance as expired while the spending path still reads the raw temporary entry. If the spender consumes the full stale allowance, the remaining allowance is written as zero, and the expired deadline does not prevent the transfer or burn.

This is broader than a single function bug: the project is pinned to an outdated, vulnerable dependency and therefore inherits other 0.5.0-era fixes as well. However, the expired allowance path is directly reachable through the USST and STBL token methods.

Exploitation requires prior approval, the allowance entry remaining available after `live_until_ledger`, and the spender consuming the full stale remaining allowance.

These are realistic conditions for protocol spenders, bridges, integrations, and users who rely on short-lived approvals.

Proof of Concept

Insert the following focused test in the USST token test suite. It only needs the extra imports shown below; the existing `addr`, `ADMIN`, and `USER1` helpers can be reused.

```
use soroban_sdk::testutils::Ledger;
use stellar_tokens::fungible::{AllowanceKey, StorageKey};
#[test]
fn expired_allowance_can_still_be_spent_when_entry_survives() {
    let env = Env::default();
    env.mock_all_auths();
    let admin = addr(&env, ADMIN);
    let owner = addr(&env, USER1);
    let contract_id = env.register(UsstToken, (admin.clone(), None::<u32>));
    let client = UsstTokenClient::new(&env, &contract_id);
    client.grant_role(&admin, &admin, &String::from_str(&env, MINTER_ROLE));
    client.mint(&admin, &owner, &1_000u128);
    env.ledger().set_sequence_number(1);
    client.approve(&owner, &admin, &100i128, &10);
    let key = StorageKey::Allowance(AllowanceKey {
        owner: owner.clone(),
        spender: admin.clone(),
    });
    env.as_contract(&contract_id, || {
        env.storage().temporary().extend_ttl(&key, 1000, 1000);
    });
    env.ledger().set_sequence_number(11);
    assert_eq!(client.allowance(&owner, &admin), 0);
    client.transfer_from(&admin, &owner, &admin, &100i128);
    assert_eq!(client.balance(&owner), 900);
}
```

The assertion before `transfer_from` shows that the token reports the approval as expired. The final balance assertion shows that the spender can still consume the full

stale allowance when the underlying temporary storage entry survives past `live_until_ledger`.

Impact

Token holders use allowance deadlines to bound how long a spender can move or burn their tokens, but a spender with a stale full allowance can still transfer USST/STBL or use `burn_from` after the holder-visible expiry. This breaks a core authorization invariant and can move user funds after the approved window.

Recommendation

Upgrade `stellar-tokens` and `stellar-macros` from 0.5.0 to at least OpenZeppelin Stellar Contracts v0.6.0, where the allowance-expiry fix was introduced. Prefer the latest compatible fixed release, such as v0.7.1, so the project also receives later 0.6.x/0.7.x fixes. Apply the upgrade consistently to USST, STBL, and YLD. If the dependency cannot be upgraded immediately, wrap allowance-consuming paths with an explicit `live_until_ledger >= env.ledger().sequence()` check before calling the base spend logic.

Status

Resolved

[M-02] AssetIssuer#deposit - Late deposits can capture accrued yield

Description

LT/PT yield is allocated to the staking set that exists when `distribute_yield()` is called, not the staking set that existed when that yield economically accrued. A user can deposit after unrealized yield has already built up in the issuer and still receive a pro-rata share of that earlier yield. A similar issue exists in the airdrop issuer path and has been reported as M-03.

Vulnerability Details

New deposits immediately mint YLD and enable staking in `contracts/asset-issuer/src/lib.rs:321-337`. Later, `distribute_yield()` values the current vault position with `fetch_forward_price(vault.asset_deposit_net)`, compares it against `vault.deposit_value_usd`, and treats the difference as distributable yield in `contracts/asset-issuer/src/lib.rs:478-524`.

That yield is then forwarded to the reward distributor, which divides the reward by the current `total_supply` in `contracts/asset-yield-distributor/src/lib.rs:275-319`. There is no checkpoint that binds accrued appreciation to the YLD holders who were already staked when the appreciation occurred. As a result, a depositor who joins after the oracle-driven gain exists but before `distribute_yield()` runs can share in yield that should belong only to incumbent holders.

This is distinct from `audit/M-3_late-airdrop-deposit.md`. There, the attacker captures pre-existing airdropped tokens already sitting in the issuer balance. Here, the attacker captures previously accrued price-based yield inferred from the issuer's current collateral valuation. The attack is realistic whenever the oracle-visible gain appears before a later keeper- or treasury-triggered distribution call.

Proof of Concept

This PoC proves the issue by showing that Bob receives part of the yield that accrued before Bob deposited.

Insert a focused integration test alongside the existing LT/PT withdrawal or lifecycle tests. Use one asset with `yield_duration > 0`, let Alice deposit first, then move the oracle price upward so the vault has positive unrealized yield, but do not call

`distribute_yield()` yet. After that, let Bob deposit, advance to a valid distribution time, and call `distribute_yield()`. Finally, compare Alice's and Bob's claimed rewards from `contracts/asset-yield-distributor/src/lib.rs:379-413`.

```
let alice_token_id = asset_issuer.deposit(&system.alice, &deposit_amount, &None);
rwa_oracle_client.set_price_data(&1_100_000_000);
// Bob joins after the gain already exists but before distribution.
let bob_token_id = asset_issuer.deposit(&system.bob, &deposit_amount, &None);
asset_issuer.distribute_yield(&system.treasury);
let bob_yield = distributor_client.claim(&system.bob, &(bob_token_id as u64));
assert!(bob_yield > 0);
```

Bob's expected reward from pre-existing accrued yield is zero. The current implementation gives Bob a material share because reward allocation uses the post-deposit staking supply.

Impact

Existing YLD holders can be diluted during favorable price moves, redirecting part of the already accrued yield to late depositors. The issue affects yield allocation rather than principal custody or direct USST backing.

Recommendation

Checkpoint accrued-but-undistributed yield before newly deposited stake can share in it. For example, reconciling pending yield before `deposit()` enables staking, or store a vault-level accumulator that advances the reward index before any new YLD position is added. New positions should start from the reward index after all previously accrued yield has been allocated.

Status

Acknowledged

[M-03] AirdropAssetIssuer#deposit - Late deposits can capture prior airdrops

Description

DLT/DPT airdrop yield is allocated to the staking set that exists when `distribute_yield()` is called, not the staking set that existed when the airdrop tokens arrived. A user can deposit after an airdrop has already been transferred to the issuer and still receive a pro-rata share of that earlier airdrop. A similar issue exists in the oracle-yield issuer path and has been reported as M-02.

Vulnerability Details

`AirdropAssetIssuer` calculates distributable yield from the issuer contract's live token balance minus accumulated fees in `contracts/airdrop-asset-issuer/src/lib.rs:171-197`. This balance-based accounting has no checkpoint that binds an airdrop to the YLD holders who were staked when the tokens arrived.

New deposits are public and immediately mint YLD plus enable staking in the reward distributor in `contracts/airdrop-asset-issuer/src/lib.rs:320-369` and `contracts/airdrop-asset-issuer/src/lib.rs:711-729`. Later, `distribute_yield()` sends the detected balance difference to the reward distributor in `contracts/airdrop-asset-issuer/src/lib.rs:538-584`. The distributor then divides that reward by the current `total_supply` in `contracts/asset-yield-distributor/src/lib.rs:275-319`, so a deposit made after the airdrop but before distribution participates in the old reward.

Frequent calls to `distribute_yield()` reduce the attack window, but the contracts only enforce a minimum interval through `yield_duration`; they do not force distribution immediately when an airdrop is received. This makes exploitation realistic whenever an airdrop receipt is observable, and distribution is keeper- or treasury-triggered rather than immediate.

Proof of Concept

This PoC proves that a depositor can receive a share of an airdrop that was already held by the issuer before the depositor joined. Insert the test below into `integration-tests/tests/test_dpt_calls.rs`; the file's existing imports are sufficient.

```

#[test]
fn test_late_deposit_captures_prior_airdrop_yield() {
    let system = TestIntegratedSystem::deploy();
    let env = &system.env;
    let register_client = register::StblRegisterClient::new(env, &system.registry);
    let benji_decimals: u32 = 6;
    let benji_token = env.register(
        usst_token::UsstToken,
        (system.admin.clone(), Some(benji_decimals)),
    );
    let benji_token_client = StblTokenClient::new(env, &benji_token);
    system.grant_minting_access(&benji_token, &system.admin);
    let asset_name = String::from_str(env, "BenJi");
    let description = String::from_str(env, "Airdrop-backed DPT asset");
    let asset_id =
        register_client.add_asset(&system.admin, &asset_name, &description, &1, &4,
    &true);
    assert_eq!(asset_id, 1);
    let rwa_oracle = env.register(rwa_oracle::RwaOracle, (system.admin.clone(),));
    let rwa_oracle_client = ExternalOracleClient::new(env, &rwa_oracle);
    rwa_oracle_client.set_price_data(&1_000_000_000);
    let oracle = env.register(
        asset_oracle::AssetOracle,
        (system.admin.clone(), rwa_oracle.clone(), &9u32, &2592000u64),
    );
    let issuer = env.register(
        AssetIssuer,
        (system.admin.clone(), system.registry.clone(), asset_id),
    );
    let issuer_access_control_client = AccessControlClient::new(env, &issuer);
    issuer_access_control_client.grant_role(
        &system.admin,
        &system.treasury,
        &String::from_str(env, YD_ROLE),
    );
    let distributor = env.register(
        asset_yield_distributor::AssetYieldDistributor,
        (system.admin.clone(), system.registry.clone(), asset_id),

```

```

);
let fees: Vec<u128> = vec![env, 10, 10, 10, 10];
register_client.setup_asset(
    &system.admin,
    &asset_id,
    &vec![
        env,
        benji_token.clone(),
        issuer.clone(),
        distributor.clone(),
        oracle.clone(),
    ],
    &10,
    &u128::pow(10, 24),
    &fees,
    &10800,
    &3600,
    &Bytes::new(env),
);
let asset_issuer = AssetIssuerClient::new(env, &issuer);
let distributor_client = YieldDistributorClient::new(env, &distributor);
let asset_vault_client = VaultClient::new(env, &issuer);
system.mint_tokens(&benji_token);
system.mint_tokens_to_user(&benji_token, &system.admin, None);
let deposit_amount: u128 = 1000u128 * u128::pow(10, benji_decimals);
benji_token_client.approve(&system.alice, &issuer, &(deposit_amount as i128),
&1000);
benji_token_client.approve(&system.bob, &issuer, &(deposit_amount as i128),
&1000);
let alice_token_id = asset_issuer.deposit(&system.alice, &deposit_amount, &None);
// The airdrop arrives before Bob has any position.
let airdrop_amount: i128 = (100u128 * u128::pow(10, benji_decimals)) as i128;
benji_token_client.transfer(&system.admin, &issuer, &airdrop_amount);
// Time passes until rewards may be distributed, but distribution has not run
yet.
system.env.ledger().set(LedgerInfo {
    sequence_number: env.ledger().sequence() + (3605 / 5) as u32,
    timestamp: env.ledger().timestamp() + 3605,

```

```

        ..system.env.ledger().get()
    });
    // Bob joins after the airdrop is already held by the issuer.
    let bob_token_id = asset_issuer.deposit(&system.bob, &deposit_amount, &None);
    asset_vault_client.distribute_yield(&system.treasury);
    let alice_yield = distributor_client.claim(&system.alice, &(alice_token_id as
u64));
    let bob_yield = distributor_client.claim(&system.bob, &(bob_token_id as u64));
    // Bob should receive no part of a pre-existing airdrop, but he does.
    assert!(bob_yield > 0);
    assert!(alice_yield > 0);
    assert!(
        bob_yield >= alice_yield / 2,
        "late depositor captured a material share: alice={}, bob={}",
        alice_yield,
        bob_yield
    );
}

```

Bob's expected reward from a pre-existing airdrop is zero. The current implementation gives Bob a material share because reward allocation uses the post-deposit staking supply.

Impact

Existing YLD holders can be diluted during material airdrop events, redirecting part of their expected yield to late depositors. The issue affects yield allocation rather than principal custody or USST backing.

Recommendation

Checkpoint airdrop rewards before newly deposited stake can share in them. For example, track a `last_accounted_balance` or `pending_airdrop_balance`, reconcile pending yield before `deposit()` enables new staking, and update accounting after distribution, withdrawals, and fee withdrawals. Alternatively, block new deposits while the uncheckpointed airdrop yield exists. New positions should start from the reward index after all previously accrued airdrop yield has been allocated.

Status

Acknowledged



[M-04] USST blacklist does not block vault redemption burns

Description

The USST blacklist prevents blacklisted accounts from receiving newly minted USST, bridge-minted USST, bridge-burning USST, or transferring USST, but it does not prevent the normal `burn` and `burn_from` paths. Vault redemption uses `burn_from` to consume the withdrawing account's USST, so a blacklisted holder can still redeem any valid YLD position they own into the underlying RWA.

Vulnerability Details

`UsstToken` explicitly applies `is_address_blacklisted` in the minting, bridge, and transfer paths, including `bridge_burn`, `transfer`, and `transfer_from`, but its burnable extension is imported through the default implementation without wrapping `burn` or `burn_from` with the same blacklist check in `contracts/usst-token/src/lib.rs:156-260`.

The redemption path is gated by YLD ownership, not by USST provenance. `AssetIssuer::internal_withdraw` reads the YLD metadata, verifies that the NFT exists, checks that the metadata `asset_id` matches the issuer's asset, and requires the withdrawing owner to be the current YLD owner in `contracts/asset-issuer/src/lib.rs:356-370`. Therefore, holding USST alone is not sufficient to redeem arbitrary vault assets. However, if the blacklisted account owns a valid YLD position and enough USST, the issuer sends the represented RWA to that account and then calls `Core.exit` in `contracts/asset-issuer/src/lib.rs:392-412`. `Core.exit` burns the user's USST through `usst_token_client.burn_from` in `contracts/core/src/lib.rs:207-214`, and this burn path does not reject blacklisted users.

As a result, blacklist enforcement is bypassed at the protocol exit point: a frozen USST holder cannot transfer USST directly, but can still use USST to redeem an owned or acquired YLD-backed RWA position.

A blacklisted user needs a valid YLD position, enough USST to cover the position's `stable_value_net`, and the required approvals for `Core`. No privileged role is required after the user owns the position.

Proof of Concept

Add an integration test alongside the existing withdrawal tests. The setup should create a normal deposit so Alice receives USST and a YLD `token_id`, then grant `LIST_MANAGER_ROLE` on USST to the test admin.

```
usst_access.grant_role(&admin, &admin, &String::from_str(env, LIST_MANAGER_ROLE));
usst.enable_blacklist(&admin, &alice);
assert!(usst.try_transfer(&alice, &bob, &1i128).is_err());
usst.approve(&alice, &core, &(stable_value_net as i128), &live_until);
yld.approve(&alice, &core, &token_id, &live_until);
let rwa_before = rwa.balance(&alice);
asset_issuer.withdraw(&alice, &token_id, &None);
assert!(rwa.balance(&alice) > rwa_before);
```

This demonstrates that the same account blocked from ordinary USST transfer can still consume USST via the unchecked redemption burn and receive the RWA represented by its YLD position.

Impact

Blacklisted users can bypass the intended freeze at redemption time and exit into RWA represented by YLD positions they own or acquire. This breaks a security and compliance control, but it does not let an attacker redeem unrelated vault assets with USST alone; the attacker must own the corresponding YLD claim.

Recommendation

Replace the default `FungibleBurnable` implementation for `UsstToken` with explicit `burn` and `burn_from` implementations that enforce the same freeze policy as other value-moving paths. At minimum, both burn paths should call `Pauser::when_not_paused(&env)` and reject `from` when `is_address_blacklisted(&env, Some(&from), None)` is true before calling `Base::burn` or `Base::burn_from`. Add a regression test that blacklists a depositor, confirms USST transfer reverts, and confirms `AssetIssuer.withdraw` also reverts without releasing RWA.

Status

Resolved



[M-05] asset-issuer/src/lib.rs#deposit_erc20 - Vault accounting trusts requested collateral instead of received collateral

Description

The deposit paths call `transfer_from()` and then update vault accounting using the requested metadata amount. They do not compare the issuer's token balance before and after the transfer.

Vulnerability Details

The standard issuer computes `token_amount` from `metadata.asset_value`, transfers that requested amount, and then records the full metadata value in vault accounting in `contracts/asset-issuer/src/lib.rs:183-219`.

```
let token_amount =
    DecimalConverter::convert_from_18_decimals(metadata.asset_value,
token_decimals);
    token_client.transfer_from(
        &env.current_contract_address(),
        &from,
        &env.current_contract_address(),
        &(DecimalConverter::convert_to_i128(&env, token_amount)),
    );
    // Calculate fee asset values
let oracle_client = AssetOracleClient::new(&env, &asset.oracle);
let deposit_fees_asset_value =
    oracle_client.fetch_inverse_price(&metadata.deposit_fee_amount);
let insurance_fee_asset_value =
    oracle_client.fetch_inverse_price(&metadata.insurance_fee_amount);
let mut vault = get_vault_data(&env);
vault.asset_deposit_gross += metadata.asset_value;
vault.asset_deposit_net +=
    metadata.asset_value - (deposit_fees_asset_value +
insurance_fee_asset_value);
    vault.deposit_value_usd += metadata.stable_value_net +
metadata.haircut_amount;
```

The airdrop issuer has the same pattern in `contracts/airdrop-asset-issuer/src/lib.rs:199-238`. The code assumes that the token transfer delivers exactly the requested amount. Fee-on-transfer tokens, rebasing behavior, malicious token implementations, or non-standard collateral contracts can deliver less than requested while still allowing the issuer to mint USST and YLD against the larger requested amount.

Impact

USST and YLD can be issued against collateral the issuer never actually received. This creates undercollateralized positions and corrupts vault accounting from the moment of deposit. The issue is most severe when an unsupported or malicious collateral token is onboarded, but the deposit code itself does not enforce exact receipt.

Recommendation

Measure the issuer's collateral-token balance before and after `transfer_from()` and use the actual received delta for pricing and vault accounting. Reject deposits when the received delta differs from the expected amount unless the protocol explicitly supports fee-on-transfer assets and prices the net received amount. Asset onboarding should also restrict collateral tokens to implementations with stable, exact-transfer semantics.

Status

Resolved

[M-06] asset-issuer/src/lib.rs#emergency_withdraw - Emergency drain leaves stale vault state reusable

Description

The `emergency_withdraw()` paths transfer the issuer contract's token balance to treasury but do not reset vault accounting, close positions, or make the asset terminal. The asset can later be re-enabled with stale state still present.

Vulnerability Details

The standard issuer emergency path in `contracts/asset-issuer/src/lib.rs:630-662` checks that the asset is inactive and that fee buckets are zero, then transfers the contract's full token balance to treasury.

```
if asset.is_active() {
    panic_with_error!(&env, CustomContractError::AssetIsActive);
}
let vault: VaultStruct = get_vault_data(&env);
let total_fees =
    vault.deposit_fees + vault.withdraw_fees + vault.yield_fees +
vault.insurance_fees;
if total_fees != 0 {
    panic_with_error!(&env, CustomContractError::UncollectedFeesExist);
}
let token_client = UsstTokenClient::new(&env, &asset.token);
let balance = token_client.balance(&env.current_contract_address());
token_client.transfer(&env.current_contract_address(), &treasury, &balance);
```

The airdrop issuer repeats the same pattern in `contracts/airdrop-asset-issuer/src/lib.rs:654-685`. Neither path clears vault fields such as `deposit_value_usd`, `asset_deposit_net`, or outstanding position state. The registry can later move a disabled asset back to enabled in `contracts/register/src/lib.rs:527-545`.

Impact

Collateral can be drained to treasury while vault accounting and outstanding positions continue to describe collateral that is no longer in the issuer. If the asset is later re-enabled, deposits, withdrawals, yield distribution, or accounting views can operate against stale state and misleading solvency assumptions.

Recommendation

Treat emergency withdrawal as a terminal asset state, or make it fully reconciling. A safe implementation should either permanently prevent re-enabling the asset after emergency drain, or atomically close all positions, clear vault accounting, update register deposits, and emit explicit terminal-state events before funds leave the issuer.

Status

Resolved

[M-07] usst-token/src/lib.rs#transfer_from - Blacklisted spenders can still receive approvals and spend from non-blacklisted accounts

Description

USST blacklist enforcement checks transfer participants but not the spender in `transfer_from()`, and `approve()` performs no blacklist validation. This leaves a gap where a blacklisted spender can still be approved and can move funds from a non-blacklisted owner to a non-blacklisted recipient.

Vulnerability Details

The USST `transfer_from()` override checks only `from` and `to` before delegating to the base implementation in `contracts/usst-token/src/lib.rs:218-223`.

```
fn transfer_from(env: &Env, spender: Address, from: Address, to: Address, amount: i128) {  
    // ---- custom rules ----  
    is_address_blacklisted(&env, Some(&from), Some(&to));  
    // ---- call base implementation ----  
    Base::transfer_from(env, &spender, &from, &to, amount);  
}
```

The approval path in `contracts/usst-token/src/lib.rs:238-240` delegates directly to the base token without checking either the owner or spender against the blacklist.

```
fn approve(env: &Env, owner: Address, spender: Address, amount: i128, live_until_ledger: u32) {  
    Base::approve(env, &owner, &spender, amount, live_until_ledger)  
}
```

Impact

A blacklisted account can retain indirect USST movement capability as a spender for non-blacklisted owners. This weakens the blacklist as a compliance or incident-response control because the listed account can still participate in allowance-based flows.

Recommendation

Apply blacklist checks consistently to all relevant approval and allowance participants. `approve()` should reject blacklisted owners and spenders, and `transfer_from()` should reject a blacklisted spender in addition to checking `from` and `to`.

Status

Resolved

Low

[L-01] `StblRegister#disable_asset` - Active deposits can be made unwithdrawable

Description

`disable_asset()` can move an enabled asset to `Disabled` without checking whether the asset still has outstanding deposits. The registry already tracks live asset deposits through `increment_asset_deposits()` and `decrement_asset_deposits()` in `contracts/register/src/lib.rs:570-595`, but `disable_asset()` does not consult this accounting before changing the asset status in `contracts/register/src/lib.rs:507-524`.

Vulnerability Details

Both issuer implementations require the asset to be active before allowing user withdrawals. The standard issuer's `assert_setup_done()` reverts when `asset_data.is_active()` is false in `contracts/asset-issuer/src/lib.rs:162-167`, and `internal_withdraw()` calls that guard before validating and settling the user's YLD position in `contracts/asset-issuer/src/lib.rs:348-412`.

The airdrop issuer repeats the same pattern: `assert_setup_done()` rejects non-active assets in `contracts/airdrop-asset-issuer/src/lib.rs:162-167`, and `internal_withdraw()` calls it at the start of the withdrawal path in `contracts/airdrop-asset-issuer/src/lib.rs:372-435`.

Expired treasury withdrawals are also blocked by the same active-asset precondition because `withdraw_expired()` calls `assert_setup_done()` before expiry settlement in `contracts/asset-issuer/src/lib.rs:727-785` and `contracts/airdrop-asset-issuer/src/lib.rs:751-805`.

Since `disable_asset()` only checks that the current status is `Enabled`, a `REGISTER_ROLE` holder can disable an asset while `fetch_deposits()` would still return a nonzero live deposit amount from `contracts/register/src/lib.rs:628-629`. This can freeze normal user exits and treasury expiry handling until the asset is re-enabled.

Proof of Concept

A focused test can reproduce the lockout with the existing setup helpers:

```
// 1. Create and set up an enabled LT or DLT asset.
// 2. Alice deposits, which increments the registry's live deposit accounting.
let token_id = asset_issuer.deposit(&alice, &deposit_amount, &None);
assert!(register.fetch_deposits(&asset_id) > 0);
// 3. A REGISTER_ROLE holder disables the asset while deposits are still live.
register.disable_asset(&admin, &asset_id);
// 4. Alice can no longer withdraw because the issuer requires AssetStatus::Enabled.
assert!(asset_issuer.try_withdraw(&alice, &token_id, &None).is_err());
// 5. The same active-asset guard also blocks expired treasury withdrawal.
assert!(asset_issuer
    .try_withdraw_expired(&treasury, &token_id)
    .is_err());
```

Impact

Users with existing YLD positions can be prevented from redeeming their underlying asset through the normal withdrawal path. For LT and DLT assets, treasury expiry processing can also be blocked. Funds are not directly stolen, and a privileged actor can later re-enable the asset, but withdrawals remain unavailable while the asset is disabled.

Recommendation

Prevent assets with live deposits from being disabled. `disable_asset()` should read the tracked deposit amount for the target asset and revert when it is nonzero before setting `asset.status = AssetStatus::Disabled`.

If the protocol needs to stop new deposits during maintenance, incident response, or delisting, introduce a separate state or flag that blocks new issuance while still allowing `withdraw()` and `withdraw_expired()` to settle existing positions. Add regression tests proving that an asset with nonzero `fetch_deposits()` cannot be disabled, and that any deposit-disabled state still permits withdrawals for already-issued YLD positions.

Status

Acknowledged



[L-02] AssetOracle - Transient price spikes are accepted without smoothing

Description

The protocol uses the current external oracle price directly without TWAP, median/quorum validation, or maximum price-deviation checks. If the external price source briefly reports an inflated fresh price, deposits use that price to calculate `stable_value_gross` and `stable_value_net` in `contracts/asset-issuer/src/lib.rs:424-466`, and Core then mints USST for the inflated `meta.stable_value_net` in `contracts/core/src/lib.rs:166-177`.

Vulnerability Details

`AssetOracle` only checks that the oracle is enabled, the reported price is nonzero, and the timestamp is fresh in `contracts/asset-oracle/src/lib.rs:232-254`. After that, `fetch_forward_price()` directly multiplies the asset amount by the current price in `contracts/asset-oracle/src/lib.rs:257-275`. The contract does not compare the new price against the previously accepted price, does not maintain a time-weighted average, and does not require agreement from multiple independent price sources.

During deposit, `generate_metadata()` stores the oracle-derived `stable_value_gross`, subtracts fees, and stores `stable_value_net` as the USST mint amount in `contracts/asset-issuer/src/lib.rs:447-466`. Therefore, a short-lived inflated price can let a depositor receive more USST for the same collateral amount. The same direct current-price dependency also affects yield accounting: `distribute_yield()` treats the difference between the current oracle valuation and `deposit_value_usd` as yield in `contracts/asset-issuer/src/lib.rs:478-498`, so a price spike can create artificial distributable yield.

The issue depends on a failure or manipulation of the external oracle feed or updater.

Proof of Concept

The behavior was verified with an integration test against the existing mock oracle. The test deposited the same collateral amount twice: first at price `1_000_000_000`, then after advancing the ledger to avoid same-ledger cache reuse, at price `2_000_000_000`. The second deposit minted exactly twice as much USST as the first deposit.

```

let deposit_amount: u128 = u128::pow(10, 20);
rwa_oracle_client.set_price_data(&1_000_000_000);
let alice_usst_before = usst_token_client.balance(&system.alice);
asset_issuer.deposit(&system.alice, &deposit_amount, &None);
let alice_usst_delta = usst_token_client.balance(&system.alice) - alice_usst_before;
env.ledger().set(LedgerInfo {
    sequence_number: env.ledger().sequence() + 1,
    timestamp: env.ledger().timestamp() + 5,
    ..env.ledger().get()
});
rwa_oracle_client.set_price_data(&2_000_000_000);
let bob_usst_before = usst_token_client.balance(&system.bob);
asset_issuer.deposit(&system.bob, &deposit_amount, &None);
let bob_usst_delta = usst_token_client.balance(&system.bob) - bob_usst_before;
assert_eq!(bob_usst_delta, alice_usst_delta * 2);

```

This PoC does not treat the mock oracle as the production vulnerability. It demonstrates that once `AssetOracle` receives a fresh but economically abnormal price, the protocol accepts it directly with no anti-spike controls.

Impact

A manipulated or erroneous fresh price can cause over-minting of USST for the same collateral amount and can also create artificial yield for distribution.

Recommendation

Add anti-spike validation to the oracle layer. The protocol should still reject stale prices, but fresh prices should also pass TWAP or median/quorum validation, per-asset sanity bounds, and a maximum-deviation check against the previously accepted price. If a fresh price exceeds the allowed movement threshold, deposits, withdrawals, and yield distribution should fail closed until a safe price is established. For yield distribution, consider using a smoothed price or delayed realization so that a single price update cannot immediately create distributable yield.

Status

Acknowledged

[L-03] AssetIssuer#withdraw - Live withdrawal fee reprices existing positions

Description

Existing YLD positions snapshot the configured withdrawal fee at deposit time, but the issuer contracts ignore that snapshot during withdrawal. `generate_metadata()` stores `asset.withdraw_fees` in the position metadata in `contracts/asset-issuer/src/lib.rs:436-459`, while `withdraw_erc20()` later calculates the actual fee from the current live asset config in `contracts/asset-issuer/src/lib.rs:253-260`.

Vulnerability Details

The same mismatch exists in the airdrop issuer: deposit metadata snapshots `asset.withdraw_fees` in `contracts/airdrop-asset-issuer/src/lib.rs:459-482`, but withdrawal computes `withdraw_fee_amount` from live `asset.withdraw_fees` in `contracts/airdrop-asset-issuer/src/lib.rs:277-284`.

`StblRegister::set_fees()` allows a `REGISTER_ROLE` holder to update `withdraw_fee` for an enabled asset in `contracts/register/src/lib.rs:397-435`. After that update, all existing positions still carry their original `metadata.fees.withdraw_fee`, but the withdrawal path charges the newly configured fee. This makes fee changes retroactive for already-issued YLD positions even though the metadata structure appears designed to preserve per-position terms.

The issue requires privileged fee configuration, so it is a governance and operator-risk issue rather than an external user bypass. However, the affected user path is ordinary withdrawal: a user who deposited under one fee schedule can receive less collateral, or potentially hit arithmetic underflow, because settlement no longer follows the terms snapshotted into their YLD metadata.

Proof of Concept

The issue can be reproduced with a focused integration test:

```
// 1. Configure an enabled asset with withdraw_fee = 0.
// 2. Alice deposits and receives a YLD position whose metadata snapshots that fee.
let token_id = asset_issuer.deposit(&alice, &deposit_amount, &None);
let metadata = yld.get_nft_data(&token_id).unwrap();
```

```

assert_eq!(metadata.fees.withdraw_fee, 0);
// 3. REGISTER_ROLE raises the live withdraw fee for the same asset.
register.set_fees(&admin, &asset_id, &0, &new_withdraw_fee, &0, &0);
// 4. Alice withdraws the old position.
// Expected: settlement uses metadata.fees.withdraw_fee == 0.
// Actual: settlement uses the live asset.withdraw_fees value.
asset_issuer.withdraw(&alice, &token_id, &None);

```

A second variant can deposit with nonzero deposit or insurance fees, raise the live withdrawal fee close to `FEES_CONSTANT`, and assert that the subtraction in the withdrawal calculation reverts when the live fee exceeds the old position's withdrawable value.

Impact

Users can be overcharged on withdrawal after a fee update, receiving less collateral than the fee terms recorded in their YLD metadata imply. In more extreme configurations, withdrawals for old positions can revert instead of settling, leaving users unable to redeem through the normal path until configuration or code is corrected.

Recommendation

Use the position's snapshotted fee terms for settlement. Both issuer implementations should calculate `withdraw_fee_amount` from `metadata.fees.withdraw_fee` instead of live `asset.withdraw_fees`. Registry fee changes should apply prospectively to newly issued YLD positions, while existing positions continue to settle under the fee schedule stored in their metadata. Add regression tests proving that updating `withdraw_fee` after deposit does not alter the withdrawal amount or block withdrawal for an existing position.

Status

Acknowledged

[L-04] AssetIssuer#modify_staking - Splitter role can forge staking balances

Description

SPLITTER_ROLE is intended to let the YLD splitter rebalance staking after a split, but both issuer implementations expose arbitrary staking updates to any holder of that role in `contracts/asset-issuer/src/lib.rs:590-624` and `contracts/airdrop-asset-issuer/src/lib.rs:497-531`.

Vulnerability Details

`modify_staking()` accepts caller-supplied token IDs, stake amounts, and enable/disable flags. It only checks that the caller has SPLITTER_ROLE, then forwards each supplied tuple to the asset's reward distributor. The issuer does not verify that each token exists, belongs to the current asset, is owned by the expected holder, or that the supplied stake equals the token's YLD metadata value.

The distributor trusts issuer calls. `enable_staking()` checks only that the caller is the registered issuer, then adds the supplied value to the token's staking balance and the global total supply in `contracts/asset-yield-distributor/src/lib.rs:328-343`. Once fake staking state exists, `claim()` pays rewards to the YLD owner after checking only ownership and metadata validity in `contracts/asset-yield-distributor/src/lib.rs:379-414`.

This is not an unprivileged cross-asset split exploit. The canonical splitter builds staking updates from the original and newly minted split metadata before calling `modify_staking()` in `contracts/yld-splitter/src/lib.rs:281-304`. The risk is excessive privilege: a malicious, compromised, or misgranted SPLITTER_ROLE holder can bypass that intended narrow flow and mutate reward accounting directly. Final severity is Low under the privileged-actor cap because exploitation depends on discretionary role abuse or key/contract compromise rather than a normal user path.

Proof of Concept

A minimal test can grant SPLITTER_ROLE to an attacker-controlled address or malicious splitter contract, mint or obtain any valid YLD token for the asset, then call `modify_staking()` with an inflated stake:

```

issuer_access_control.grant_role(&admin, &attacker, &String::from_str(env,
SPLITTER_ROLE));
let token_ids = vec![env, attacker_token_id];
let stakes = vec![env, 1_000_000_000_000_000_000_000u128];
let flags = vec![env, true];
asset_issuer.modify_staking(&attacker, &token_ids, &stakes, &flags);
// After a normal yield distribution, the forged stake receives a disproportionate
reward.
let reward = distributor.claim(&attacker_owner, &(attacker_token_id as u64));
assert!(reward > 0);

```

The same test should assert that `fetch_staking_data(attacker_token_id).balance` and `fetch_total_supply()` increase by the caller-supplied value even though no deposit or valid split created that stake.

Impact

Distributor total supply can be inflated, honest stakers can be diluted, and future yield can be routed to attacker-controlled YLD tokens. The issue affects reward distribution accounting, not principal collateral custody.

Recommendation

Reduce `SPLITTER_ROLE` to the narrow operation it is meant to perform. The issuer should not accept arbitrary stake values from the role holder. Instead, `modify_staking()` should verify every token's YLD metadata, require `metadata.asset_id == get_asset_id()`, derive each stake internally as `stable_value_net + haircut_amount`, and enforce that the update corresponds to a valid split or merge operation.

As defense in depth, the distributor should reject staking updates and claims when the YLD metadata asset id does not match the distributor's asset id. Keep `SPLITTER_ROLE` assigned only to audited splitter contracts, and protect grants and upgrades of those contracts with multisig and timelock controls.

Status

Acknowledged

[L-05] AssetIssuer#withdraw_expired - Stale expired YLD remains transferable

Description

Expired positions are not finalized consistently after treasury confiscates their collateral. The LT `withdraw_expired()` flow transfers the underlying collateral to treasury and disables staking, but it does not burn, disable, or transfer the corresponding YLD NFT in `contracts/asset-issuer/src/lib.rs:727-785`. The same DLT expiry flow exists in `contracts/airdrop-asset-issuer/src/lib.rs:751-809`.

Vulnerability Details

The expired withdrawal path loads the YLD metadata, checks that the token belongs to the current asset, and checks that the configured duration has passed. It then calls `withdraw_erc20()` to transfer collateral to treasury and `disable_staking()` to remove the token's staking balance, but it never changes the YLD token state in `contracts/asset-issuer/src/lib.rs:741-784`.

The YLD transfer hook only checks whether metadata exists and `is_disabled` is false before delegating to the NFT base transfer in `contracts/yld-token/src/lib.rs:83-94` and `contracts/yld-token/src/lib.rs:282-288`. Since `withdraw_expired()` leaves `is_disabled` unchanged, the user keeps an active-looking transferable YLD after the collateral represented by that YLD has already been confiscated.

The stale YLD does not appear to retain on-chain economic value. Normal LT withdrawal rejects after the configured duration has passed in `contracts/asset-issuer/src/lib.rs:378-387`. Future rewards should also be unavailable because `withdraw_expired()` disables staking, and reward calculation returns zero once the staking balance is zero in `contracts/asset-yield-distributor/src/lib.rs:242-247`. This occurs on the normal successful `withdraw_expired()` path. No user cooperation, privileged key compromise, or unusual market condition is required once an LT or DLT position expires and treasury finalizes it.

Proof of Concept

Insert the following test into `integration-tests/tests/test_withdraw_flow.rs`. It uses the existing `run_setup_till_first_deposit()` helper from that file to create an LT asset

and first deposit, advances past expiry, lets treasury confiscate the collateral, and then shows that Alice still owns and can transfer the non-disabled YLD.

```
#[test]
fn test_withdraw_expired_leaves_user_with_active_transferable_yld() {
    let (system, issuer) = run_setup_till_first_deposit(1);
    let env = &system.env;
    let register_client = register::StblRegisterClient::new(env, &system.registry);
    let asset_issuer = AssetIssuerClient::new(env, &issuer);
    let asset_id: u64 = asset_issuer.fetch_asset_id();
    let asset_definition: AssetDefinition =
register_client.fetch_asset_data(&asset_id).unwrap();
    let benji_token_client = StblTokenClient::new(env, &asset_definition.token);
    let yld_nft_client = YldTokenClient::new(env, &system.yld);
    let first_deposit_id: u32 = 1;
    system.env.ledger().set(LedgerInfo {
        sequence_number: env.ledger().sequence() + ((4 * DAY) / 5) as u32,
        timestamp: env.ledger().timestamp() + (4 * DAY) + 1,
        ..system.env.ledger().get()
    });
    let treasury_balance_before = benji_token_client.balance(&system.treasury);
    asset_issuer.withdraw_expired(&system.treasury, &first_deposit_id);
    assert!(benji_token_client.balance(&system.treasury) > treasury_balance_before);
    let metadata_after_expiry =
yld_nft_client.get_nft_data(&first_deposit_id).unwrap();
    assert_eq!(metadata_after_expiry.is_disabled, false);
    assert_eq!(yld_nft_client.owner_of(&first_deposit_id), system.alice);
    yld_nft_client.transfer(&system.alice, &system.bob, &first_deposit_id);
    assert_eq!(yld_nft_client.owner_of(&first_deposit_id), system.bob);
}
```

Run it with:

```
cargo test -p integration-tests
test_withdraw_expired_leaves_user_with_active_transferable_yld
```

Impact

A user can retain and transfer a YLD NFT that still appears active after the collateral backing that position has been confiscated by treasury. This can mislead users and integrations that treat YLD ownership or `is_disabled == false` as evidence of an active position, and it leaves protocol state inconsistent after an expiry event.

Recommendation

Make expiry finalize the YLD state atomically with collateral confiscation. Both LT and DLT `withdraw_expired()` implementations should either burn the expired YLD, mark its metadata disabled/expired, or transfer the YLD to treasury according to the protocol's intended ownership model. YLD transfer logic should reject expired positions so a user cannot continue to circulate an active-looking token after its collateral has been confiscated.

Status

Resolved

[L-06] Asset onboarding - Token assumptions are not enforced

Description

The asset onboarding flow accepts configured asset metadata and token addresses without enforcing the token assumptions that issuer logic later relies on. There are two related gaps:

1. Token decimals are trusted at runtime. Deposits read `decimals()` from the configured token before normalizing the user-supplied amount to 18-decimal accounting units in `contracts/asset-issuer/src/lib.rs:432-434`.
2. Asset type metadata can imply support beyond the implemented fungible-token issuer flow. The type model includes `ERC20`, `ERC721`, and multiple contract types in `libraries/stbl-core-types/src/asset_definition.rs:22-40`, but the issuer deposit and withdrawal paths use fungible token calls such as `decimals()`, `transfer_from()`, and `transfer()` in `contracts/asset-issuer/src/lib.rs:183-197` and `contracts/asset-issuer/src/lib.rs:262-273`.

Vulnerability Details

1. Token decimals are trusted at runtime

`setup_asset()` accepts the asset token address from `REGISTER_ROLE` and only rejects component addresses that equal the registry itself in `contracts/register/src/lib.rs:317-338`. The issuer later trusts that token contract's live `decimals()` response when generating YLD metadata, pricing the deposit, and determining the USST mint amount. It then calls `decimals()` again when converting the normalized asset amount back into raw token units for the actual `transfer_from()` in `contracts/asset-issuer/src/lib.rs:183-197`.

This is safe for Stellar Asset Contract assets and for immutable audited SEP-41 tokens whose decimal metadata cannot change. However, SEP-41 compatibility alone does not prove immutability. A contract token can implement `decimals()` from mutable storage, return dynamic values, or be upgradeable by its own issuer. If such a token is onboarded, the protocol can account for one scale while the token transfers another scale, causing collateral, fees, or USST issuance to diverge from the actual token units.

2. Asset type metadata can imply support beyond the implemented issuer flow

`add_asset()` accepts raw `asset_type` and `contract_type` codes and stores the resulting

enum values in `contracts/register/src/lib.rs:202-232`. The enum surface includes `ERC721` and contract-type variants that can make unsupported asset categories appear first-class in `libraries/stbl-core-types/src/asset_definition.rs:22-40`. However, the standard issuer flow is fungible-token shaped: it reads token decimals, transfers raw fungible token amounts into the issuer, and later transfers raw fungible token amounts back out through the withdrawal path in `contracts/asset-issuer/src/lib.rs:262-273`. Neither gap means the registration function can prove every arbitrary token property on-chain. Some assumptions, such as decimal immutability and supported asset class, must be established by asset policy, allowlisting, implementation review, or explicit on-chain guards before enabling the asset.

Proof of Concept

1. Token decimals are trusted at runtime

This can be reproduced with a SEP-41-compatible mock token whose `decimals()` value is mutable:

```
// 1. Register the mock token as the asset token while it reports 6 decimals.
mock_token.set_decimals(6);
register.setup_asset(..., mock_token.address(), issuer, distributor, oracle, ...);
// 2. Alice deposits 1_000_000 raw units. The issuer normalizes using 6 decimals.
asset_issuer.deposit(&alice, &1_000_000u128, &None);
// 3. The token owner changes decimals before a later accounting path.
mock_token.set_decimals(18);
// 4. Later conversions use the new scale, so recorded 18-decimal accounting
// and raw token movement no longer describe the same asset amount.
asset_issuer.withdraw(&alice, &token_id, &None);
```

The important assertion is that the recorded YLD metadata and vault accounting are derived from the token's first decimal value, while later raw token movement can be derived from a different value.

2. Asset type metadata can imply support beyond the implemented issuer flow

This can be reproduced by registering an asset with metadata that implies an unsupported asset class, such as `asset_type = ERC721`, while the issuer still attempts the fungible-token deposit and withdrawal flow:



```
let asset_id = register.add_asset(&admin, &name, &description, &2, &1, &false);
register.setup_asset(..., token, issuer, distributor, oracle, ...);
// The configured metadata says ERC721, but the issuer still expects
// fungible decimals and amount-based transfer methods.
asset_issuer.deposit(&alice, &amount, &None);
```

Impact

Deposits, withdrawals, fees, or USST issuance can be mis-scaled relative to the actual token units. This can lead to incorrect collateral accounting, over- or under-minting, failed withdrawals, or operational recovery work for the affected asset.

Deposits and withdrawals can fail, or integrations can misinterpret YLD positions as representing an asset class the issuer does not actually support.

Recommendation

Restrict onboarding to SAC assets or to pre-approved SEP-41 token implementations whose decimal metadata and upgrade controls have been reviewed under the protocol's trust model. For each non-SAC candidate, review whether `decimals()` can be changed through token admin setters, mutable metadata storage, or token Wasm upgrades controlled outside protocol governance.

Document the accepted token classes and make decimal stability an explicit onboarding requirement. If the protocol wants an on-chain guard, store the expected decimals as an assertion and reject future asset operations if the token reports a different value. Treat that guard as change detection only, not as proof that an arbitrary token's decimals are immutable.

Also document and enforce the supported asset-type matrix. If the current issuer contracts support only fungible SEP-41/SAC-like assets, reject unsupported `asset_type` and `contract_type` combinations during asset creation or setup, or remove unimplemented categories from the public configuration surface until support exists.

Status

Resolved

[L-07] Register - Yield fee can be raised to 100%

Description

REGISTER_ROLE can configure `yield_fee` up to `FEES_CONSTANT`, while the aggregate fee cap excludes `yield_fee` from the total checked in `contracts/register/src/lib.rs:287-308` and `contracts/register/src/lib.rs:417-426`.

Vulnerability Details

The registry validates each fee field individually, so `yield_fee == FEES_CONSTANT` is accepted. However, the aggregate check only sums `deposit_fee + insurance_fee + withdraw_fee + cut`, leaving yield fees independent of the cap.

This also affects already-open positions because future yield distributions read the live asset definition rather than a fee value snapshotted into the YLD metadata. `distribute_yield()` fetches the current asset and calls `asset.calculate_yield_fee(diff)` in `contracts/asset-issuer/src/lib.rs:482-495`, and the airdrop issuer follows the same pattern in `contracts/airdrop-asset-issuer/src/lib.rs:541-555`. When `yield_fees` equals `FEES_CONSTANT`, `calculate_yield_fee()` returns zero net yield and the full amount as protocol fee in `libraries/stbl-core-types/src/asset_definition.rs:257-261`.

Exploitation depends on malicious governance, privileged key compromise, or discretionary centralized-operator fee abuse outside normal protocol operation.

Proof of Concept

1. Users deposit while `yield_fee` is set below 100%.
2. A REGISTER_ROLE holder calls `set_fees()` for the enabled asset with `yield_fee = FEES_CONSTANT`.
3. After positive yield accrues, the issuer calls `distribute_yield()`.
4. The live asset configuration causes `calculate_yield_fee()` to classify all yield as fee, so no net yield is approved for the reward distributor.

Impact

Existing depositors' future yield can be fully diverted into protocol fee accounting, even though they entered the position under a lower yield fee. Principal collateral and USST balances are not directly seized by this issue.

Recommendation

Include `yield_fee` in the aggregate fee validation in both `setup_asset()` and `set_fees()`, or enforce a separate maximum that is below 100%. To protect existing positions from later repricing, snapshot yield fee terms into position metadata or require a timelocked fee-change process with clear notice before the new fee applies.

Status

Acknowledged

[L-08] Token pause - Standard token movement remains live

Description

The token contracts expose an emergency pause role, but pausing only blocks selected privileged operations and does not block standard token movement. USST checks `Pauser::when_not_paused()` for minting and bridge mint/burn in `contracts/usst-token/src/lib.rs:157-190`, while `transfer()`, `transfer_from()`, and `approve()` continue to delegate directly to the base token implementation in `contracts/usst-token/src/lib.rs:209-239`.

Vulnerability Details

This is a partial pause model rather than a global token freeze. That may be a deliberate design choice, since freezing all transfers can deny service to ordinary holders, but it weakens the pause role as an incident-containment control. STBL follows the same pattern: minting and bridge operations check pause in `contracts/stbl-token/src/lib.rs:81-123`, but the contract uses default fungible and burnable implementations in `contracts/stbl-token/src/lib.rs:129-137`, leaving standard transfers, approvals, burns, and burn-from operations outside the pause gate. YLD is the clearest incident-response concern because it represents position and redemption rights. YLD checks pause for minting, privileged burning, disabling, and enabling in `contracts/yld-token/src/lib.rs:197-261`, but `transfer()`, `transfer_from()`, `approve()`, and `approve_for_all()` remain callable while paused in `contracts/yld-token/src/lib.rs:283-319`. As a result, if a position owner, approved operator, or compromised account needs to be contained during an incident, pausing YLD does not stop the position from being moved.

The safer global-pause model is common in established token implementations. OpenZeppelin's Stellar fungible token example applies the paused-state check to state-changing token movement and approval methods, including `transfer`, `transfer_from`, `burn`, `burn_from`, and `approve`. Circle's USDC/EURC stablecoin implementation, Paxos' PYUSD/BUSD/USDP implementations, and Gemini's GUSD documentation also describe emergency pause controls that can stop transfers or approvals during critical incidents.

The issue is mainly relevant after a separate incident has already occurred and

operators are relying on pause for containment. In normal operation, keeping plain transfers live during a mint or bridge pause may be intentional.

Proof of Concept

The following minimal test pattern demonstrates the behavior for each affected token method. After pausing the token, a standard movement method still succeeds because it does not call `Pauser::when_not_paused()`.

```
client.pause(&admin);
assert_eq!(client.is_paused(), true);
// Expected under a global emergency freeze: this should panic while paused.
// Actual: transfer succeeds because the standard token path does not check pause.
client.transfer(&alice, &bob, &amount);
```

The same pattern applies to USST `approve()` and `transfer_from()`, STBL default fungible and burnable methods, and YLD `transfer()`, `transfer_from()`, `approve()`, and `approve_for_all()`.

Impact

The emergency pause cannot fully immobilize protocol assets or YLD positions during a compromise, bridge incident, legal freeze, or other critical response window. Tokens or redemption positions can continue moving while operators believe the token is paused.

Recommendation

Decide that the emergency pause is a global value-movement freeze and implement that invariant consistently. USST should check `Pauser::when_not_paused()` in `transfer`, `transfer_from`, `approve`, and `burnable` paths. STBL should override the default fungible and burnable methods so transfers, approvals, burns, and `burn_from` all check pause before calling the base implementation. YLD should check pause in `transfer`, `transfer_from`, `approve`, and `approve_for_all`, because these operations move or delegate position rights. Add paused-state tests for each affected method and update UI/docs to state that pause freezes value movement, while read-only methods remain available.

Status

Acknowledged



[L-09] AccessControl - Admin roles can be orphaned by mistake

Description

AccessControl allows the current admin of a role to replace that role's admin with any role name in `libraries/access-control/src/lib.rs:90-94`. The library does not check whether the new admin role has any holder, so an operator can accidentally assign control of a critical role to an empty role and lose the ability to manage that role through the normal RBAC flow.

Vulnerability Details

This is an admin recoverability risk rather than an external access-control bypass. The caller must still authenticate and hold the current admin role before `update_role_admin()` succeeds in `libraries/access-control/src/lib.rs:90-94`. However, once the admin role is changed, future grants and revokes require the new admin role because `require_admin_role()` resolves the role admin and checks membership in `libraries/access-control/src/lib.rs:159-164`.

The test suite demonstrates the failure mode. `DEFAULT_ADMIN_ROLE` is initially self-administered, but the admin can change its admin role to `MINTER_ROLE`; a later attempt to restore `DEFAULT_ADMIN_ROLE` as its own admin fails when the caller does not hold `MINTER_ROLE` in `libraries/access-control/src/test.rs:93-103`.

A related operator mistake exists for self-renunciation. `renounce_role()` only checks that the caller is renouncing their own role and currently holds it in `libraries/access-control/src/lib.rs:144-148`. It does not prevent the last holder of a critical role from renouncing, and the test confirms the sole holder can renounce `DEFAULT_ADMIN_ROLE` in `libraries/access-control/src/test.rs:117-129`.

The most plausible trigger is operator mistake by an already privileged account, or abuse after privileged-key compromise.

Proof of Concept

The issue can be reproduced with the existing `test_invalid_setup_role_2` and `test_renounce_role_access` flows:

```
// The sole DEFAULT_ADMIN_ROLE holder changes DEFAULT_ADMIN_ROLE's admin
```

```
// to MINTER_ROLE, then cannot restore it without holding MINTER_ROLE.
client.update_role_admin(&user1, &default_admin_role, &minter_role);
client.update_role_admin(&user1, &default_admin_role, &default_admin_role);
// The sole DEFAULT_ADMIN_ROLE holder can also renounce the role.
client.renounce_role(&user1, &user1, &default_admin_role);
assert_eq!(client.has_role(&user1, &default_admin_role), false);
```

Impact

A mistaken admin change or last-holder renounce can block future role recovery, upgrades, emergency actions, or other privileged maintenance until an out-of-band migration or contract upgrade is available.

Recommendation

Add recoverability invariants for critical roles by making replacement mandatory when the last holder would otherwise be removed. For `DEFAULT_ADMIN_ROLE` and other critical admin roles, reject direct `renounce_role()` or `revoke_role()` calls that would leave the role with no holder, and require operators to use an explicit role-transfer flow instead.

The transfer flow should be two-step: the current holder proposes a replacement, the replacement accepts and proves control of the receiving address, and only then can the previous holder be removed. Apply the same principle to `update_role_admin()` by requiring a two-step handoff or another explicit acceptance flow before a critical role's admin is changed to a different role.

Status

Acknowledged

[L-10] AccessControl - Role memberships can archive independently

Description

AccessControl stores each role membership as a persistent `RoleUser(role, account)` entry in `libraries/access-control/src/lib.rs:16-23`. These entries are written through `env.storage().persistent().set()` in `libraries/access-control/src/lib.rs:102-105` and refreshed only when the exact role/account key is read by `has_role()` in `libraries/access-control/src/lib.rs:56-64`.

Vulnerability Details

Persistent role memberships have their own TTL and can archive independently from the contract instance. The shared TTL policy extends persistent entries to about 150 days in `libraries/stbl-core-types/src/utils.rs:8-19`, but it only runs when the specific storage key is touched. As a result, rarely used roles can become archived even while the contract and other storage remain live.

All protocol roles are affected because role names are ultimately stored under the same persistent membership key. This includes `DEFAULT_ADMIN_ROLE`, `BRIDGE_ROLE`, `PAUSE_ROLE`, `MINTER_ROLE`, `LIST_MANAGER_ROLE`, `REGISTER_ROLE`, `UPGRADER_ROLE`, `EMERGENCY_ROLE`, `YLD_SPLITTER_ROLE`, and `YIELD_DISTRIBUTOR_ROLE`, which are defined in `libraries/stbl-core-types/src/lib.rs:8-17`. The most relevant liveness risk is for roles that are intentionally rarely used, such as upgrade and emergency roles. For example, asset issuer upgrades depend on the registry's `UPGRADER_ROLE` check in `contracts/asset-issuer/src/lib.rs:126-134`, and emergency withdrawals depend on `EMERGENCY_ROLE` in `contracts/asset-issuer/src/lib.rs:630-632`.

This is not a permanent loss of authorization data. Archived persistent entries can be restored by Soroban archival tooling when the transaction footprint includes the archived role entry. However, if operational scripts are not archival-aware, a time-sensitive admin, upgrade, or emergency transaction can fail first and require an additional restoration/simulation flow before it can execute.

Critical but rare roles can remain unused longer than the persistent TTL.

Proof of Concept

The issue follows from the storage layout and authorization path:



```
// Role grants create a persistent key for exactly (role, account).
env.storage().persistent().set(&DataKey::RoleUser(role, account), &true);
// Later authorization depends on reading that same persistent key.
let has_role = AccessControl::has_role(env, caller, role);
if !has_role {
    panic_with_error!(env, CustomContractError::MissingRole);
}
```

If no transaction touches a rare role membership before its persistent TTL expires, the next permissioned call must restore that archived key before `has_role()` can succeed.

Impact

Emergency, upgrade, registration, blacklist, mint, bridge, pause, splitter, or yield-distribution actions may fail or require an additional archival restoration step at the moment operators need them.

Recommendation

Move bounded protocol RBAC memberships to instance storage so role liveness follows the contract instance rather than each independent persistent membership key. Because the protocol has a small fixed role set, instance storage is appropriate for critical roles such as `DEFAULT_ADMIN_ROLE`, `UPGRADER_ROLE`, `EMERGENCY_ROLE`, `REGISTER_ROLE`, `MINTER_ROLE`, `BRIDGE_ROLE`, `PAUSE_ROLE`, `LIST_MANAGER_ROLE`, `YLD_SPLITTER_ROLE`, and `YIELD_DISTRIBUTOR_ROLE`.

If persistent storage is intentionally retained, ship archival-aware operational scripts for every permissioned function. These scripts should always simulate the invocation, detect archived role membership entries in the footprint, include the required restoration data or run a restore-footprint transaction, and retry with sufficient fees. Maintain a role-holder inventory from grant/revoke events so operators can proactively extend or restore critical role keys before incident-response use.

Status

Acknowledged

[L-11] asset-oracle/src/lib.rs#set_price_decimals - Oracle configuration can brick or misprice price-dependent flows

Description

The asset oracle accepts unsafe decimal and threshold configuration and synchronously depends on an external oracle without a fallback path. A bad configuration or external oracle failure can halt or misprice every protocol flow that needs asset pricing.

Vulnerability Details

The constructor stores the external oracle address, price decimals, and price threshold without bounding either configuration value in `contracts/asset-oracle/src/lib.rs:136-150`. The admin setters likewise accept arbitrary `price_decimals` and `price_threshold` in `contracts/asset-oracle/src/lib.rs:202-218`.

```
fn set_price_decimals(env: Env, caller: Address, decimals: u32) {
    caller.require_auth();
    AccessControl::require_role(&env, &caller, &String::from_str(&env,
DEFAULT_ADMIN_ROLE));
    set_price_decimals(&env, &decimals);
    env.events()
        .publish_event(&OracleSetDecimalsEvent { decimals });
}

fn set_price_threshold(env: Env, caller: Address, threshold: u64) {
    caller.require_auth();
    AccessControl::require_role(&env, &caller, &String::from_str(&env,
DEFAULT_ADMIN_ROLE));
    set_price_threshold(&env, &threshold);
    env.events()
        .publish_event(&OracleSetPriceThresholdEvent { threshold });
}
```

Price reads call the configured external oracle directly in `contracts/asset-oracle/src/lib.rs:232-254`. If that call fails, there is no fallback price or graceful degradation path. Conversions then use `u128::pow(10, price_decimals)`

directly in `contracts/asset-oracle/src/lib.rs:257-275` and `contracts/asset-oracle/src/lib.rs:303-344`.

```
let value =
    asset_value_with_decimals.div(&U256::from_u128(&env, u128::pow(10,
    price_decimals)));
```

For large decimals, $10^{\text{price_decimals}}$ can overflow, panic, or wrap depending on build profile. A zero threshold also makes normal reads stale immediately because the freshness check treats `timestamp + 0 <= now` as stale.

Impact

Bad oracle configuration can halt deposits, withdrawals, yield distribution, and any price-dependent accounting. Unsafe decimal settings can produce incorrect conversions or runtime traps, while external oracle downtime propagates directly to protocol liveness.

Recommendation

Reject unsafe oracle settings at construction and in setters. Bound `price_decimals` to a range that cannot overflow `u128::pow`, reject `price_threshold == 0`, and perform freshness arithmetic with checked operations. Add a fallback or explicit pause mode for external oracle failures, and add release-mode tests for decimal and threshold boundaries.

Status

Acknowledged

[L-12] core/src/lib.rs#exit - Protocol exit depends on user approvals

Description

`Core::exit()` burns both USST and YLD through approval-gated paths. A withdrawal can therefore fail unless the user has pre-approved Core to burn the USST amount and the YLD NFT.

Vulnerability Details

The issuer withdrawal path calls `Core::exit()` after vault withdrawal logic. `Core::exit()` then calls `USST.burn_from()` and `YLD.burn()` in `contracts/core/src/lib.rs:190-225`.

```
// burn USST and YLD
let usst = get_usst_address(&env);
let usst_token_client = UsstTokenClient::new(&env, &usst);
let _: () = usst_token_client.burn_from(
    &env.current_contract_address(),
    &from,
    &(DecimalConverter::convert_to_i128(&env, value)),
);
let yld = get_yld_address(&env);
let yld_token_client = YldTokenClient::new(&env, &yld);
let _: () = yld_token_client.burn(&env.current_contract_address(), &from,
&token_id);
```

USST uses the default burnable implementation in `contracts/usst-token/src/lib.rs:258-260`, so `burn_from()` follows the underlying allowance model. The YLD burn wrapper in `contracts/yld-token/src/lib.rs:217-229` also delegates to `Base::burn_from()`.

```
Base::burn_from(&env, &caller, &from, token_id);
```

This means the protocol exit path relies on user approvals that are not established by the deposit flow and are not preflight-checked by the issuer before attempting withdrawal.

Impact

Otherwise valid withdrawals can revert unless the user has granted the exact approvals Core needs. This creates a brittle redemption flow and can break integrations that assume ownership of the YLD position and sufficient USST balance are enough to exit.

Recommendation

Redesign exit so protocol-owned issuer or Core authority can burn the position created by the protocol without a separate post-deposit user approval, or add explicit preflight checks that verify USST allowance and YLD approval before any other settlement work. Document the approval requirement if it is intentionally retained.

Status

Acknowledged

[L-13] register/src/lib.rs#set_limits - Admin setters permit self-DoS configurations

Description

Several registry configuration setters accept values that can block deposits or disable expected timing behavior. The issue is permissioned, but the code does not enforce basic cross-field invariants.

Vulnerability Details

The `set_limits()` accepts `limit` and `min_deposit_amount` without checking that the limit is nonzero or that `min_deposit_amount <= limit` in `contracts/register/src/lib.rs:379-394`.

```
asset.limit = limit;
asset.min_deposit_amount = min_deposit_amount;
set_asset(&env, &asset);
```

The `set_durations()` checks only `yield_duration > duration` in `contracts/register/src/lib.rs:445-467`, so zero-duration values remain valid.

```
if yield_duration > duration {
    panic_with_error!(&env, CustomContractError::InvalidDuration);
}
asset.duration = duration;
asset.yield_duration = yield_duration;
```

Impact

A misconfigured or compromised `REGISTER_ROLE` holder can block all new deposits for an asset, make the configured minimum impossible to satisfy, or remove intended yield timing separation. These states may look like ordinary configuration changes but can deny service to users and operators.

Recommendation

Add cross-field validation to registry setters. Reject `limit == 0` for enabled assets unless a separate "deposits disabled" state is intended, require `min_deposit_amount <= limit`, and define minimum acceptable duration values. If operators need to pause deposits, expose an explicit deposit-paused state that does not overload economic parameters.

Status

Resolved

[L-14] asset-oracle/src/lib.rs#fetch_forward_price - Nonzero oracle conversions can round to zero

Description

The oracle's integer conversions do not reject cases where a nonzero input rounds down to zero. This can silently erase small economic values during forward or inverse price conversion.

Vulnerability Details

Forward conversion multiplies by the current price and divides by $10^{\text{price_decimals}}$ in `contracts/asset-oracle/src/lib.rs:271-275`.

```
let asset_value_with_decimals =
    U256::from_u128(&env, current_price).mul(&U256::from_u128(&env,
asset_amount));
let value =
    asset_value_with_decimals.div(&U256::from_u128(&env, u128::pow(10,
price_decimals)));
value.to_u128().unwrap()
```

Inverse conversion divides by the current price in `contracts/asset-oracle/src/lib.rs:318-321`.

```
let asset_value_with_decimals = U256::from_u128(&env, asset_value)
    .mul(&U256::from_u128(&env, u128::pow(10, price_decimals)));
let value = asset_value_with_decimals.div(&U256::from_u128(&env,
current_price));
value.to_u128().unwrap()
```

The batched inverse path repeats the same operation in `contracts/asset-oracle/src/lib.rs:339-343`.

```
for asset_value in asset_values {
    let asset_value_with_decimals = U256::from_u128(&env, asset_value)
        .mul(&U256::from_u128(&env, u128::pow(10, price_decimals)));
```

```
let value = asset_value_with_decimals.div(&U256::from_u128(&env,  
current_price));  
forward_prices.push_back(value.to_u128().unwrap());
```

The code does not check whether `asset_amount > 0` or `asset_value > 0` produced a zero result.

Impact

Small deposits, withdrawals, fees, or accounting values can be rounded to zero without an explicit error. This can create zero-value settlement behavior, skipped fees, or misleading accounting for boundary-sized positions.

Recommendation

Define minimum supported amounts after oracle conversion and reject nonzero inputs that would convert to zero in economically meaningful paths.

Status

Acknowledged

[L-15] stbl-core-types/src/asset_definition.rs#YLDMetadata - Stored protocol structs lack schema versions

Description

Long-lived protocol structs are stored without explicit schema versions. Future upgrades that change field layout or semantics can make old storage ambiguous or incompatible.

Vulnerability Details

The `YLDMetadata` is stored without a version field in `libraries/stbl-core-types/src/asset_definition.rs:124-141`.

```
#[contracttype]
#[derive(Clone)]
pub struct YLDMetadata {
    pub asset_id: u64,           // uint256
    pub uri: String,            // string
    pub asset_value: u128,       // uint256
    pub stable_value_gross: u128, // uint256
    pub stable_value_net: u128,  // uint256
    pub deposit_timestamp: u64,  // uint256
    pub deposit_fee_amount: u128, // uint256
    pub haircut_amount: u128,     // uint256
    pub haircut_amount_asset_value: u128, // uint256
    pub withdraw_fee_amount: u128, // uint256
    pub insurance_fee_amount: u128, // uint256
    pub fees: FeeStruct,         // FeeStruct
    pub additional_buffer: Bytes, // bytes
    pub is_disabled: bool,       // bool
}
```

The `AssetDefinition` is also versionless in `libraries/stbl-core-types/src/asset_definition.rs:188-217`, and the stored staking and vault structs are versionless in `libraries/stbl-core-types/src/asset_definition.rs:223-249`.


```

#[contracttype]
#[derive(Clone)]
pub struct StakingStruct {
    pub balance: u128,
    pub reward_index: u128,
    pub earned: u128,
    pub is_active: bool,
}
// =====
// VaultStruct
// =====
#[contracttype]
#[derive(Clone)]
pub struct VaultStruct {
    pub deposit_fees: u128,
    pub withdraw_fees: u128,
    pub insurance_fees: u128,
    pub yield_fees: u128,
    pub cumulative_hair_cut_value: u128,
    pub deposit_value_usd: u128,
    pub asset_deposit_gross: u128,
    pub asset_deposit_net: u128,
}

```

Because these records are persisted across contract upgrades, any future field addition, removal, reordering, or semantic change must be perfectly coordinated with migration code.

Impact

An upgrade can misinterpret existing storage, corrupt position metadata, or break vault and staking accounting if schemas evolve without a reliable migration discriminator. The risk grows as more assets and YLD positions exist across multiple upgrade cycles.

Recommendation

Add explicit schema version fields to long-lived stored records and write migrations that branch on the stored version. Upgrade procedures should include compatibility tests

that read pre-upgrade records after installing the new WASM and verify that all fields retain their intended meaning.

Status

Acknowledged



QA

[Q-01] AssetIssuer - Extreme-value arithmetic can misstate fees and accounting

Description

Fee calculations multiply a gross USD value by a fee before dividing by `FEES_CONSTANT`. The standard issuer computes deposit fee, haircut, insurance fee, and withdraw fee this way in `contracts/asset-issuer/src/lib.rs:452-466`, and the airdrop issuer repeats the same pattern in `contracts/airdrop-asset-issuer/src/lib.rs:475-489`. The same extreme-value arithmetic pattern appears in yield-fee calculation and vault accounting: `calculate_yield_fee()` uses saturating arithmetic in `libraries/stbl-core-types/src/asset_definition.rs:258-261`, while issuer vault fields are updated with unchecked additions and subtractions in `contracts/asset-issuer/src/lib.rs:208-216` and `contracts/asset-issuer/src/lib.rs:276-281`.

Vulnerability Details

Each fee is individually capped at `FEES_CONSTANT`, but the code does not prove that `stable_value_gross * fee` fits in `u128`. If the product overflows before division, the computed fee can be much smaller than the configured percentage. The standard withdraw path also recalculates the current withdraw fee with the same multiplication in `contracts/asset-issuer/src/lib.rs:253-258`.

Yield-fee calculation avoids panics by using `saturating_mul()` and `saturating_sub()`, but that also means an overflowing `amount * yield_fees` is silently capped rather than rejected. The resulting `(yield_usd, yield_fee_usd)` values are then used to update issuer fee balances and asset accounting in `contracts/asset-issuer/src/lib.rs:493-502`.

The vault accounting updates have a similar hardening gap: very large normalized asset values or accumulated fee balances can overflow or underflow `u128` fields instead of reverting with a protocol error. The airdrop issuer repeats the same unchecked vault updates in `contracts/airdrop-asset-issuer/src/lib.rs:228-236` and

contracts/airdrop-asset-issuer/src/lib.rs:299-304.

For a 10% fee, overflow requires `stable_value_gross > u128::MAX / 100_000_000`, or roughly \$3.4 trillion of 18-decimal USD-equivalent value. This makes the issue impractical for conventional asset limits, but it is still a fragile arithmetic edge if future assets, oracle values, decimals, or limits approach that range.

Proof of Concept

```
let gross = (u128::MAX / 100_000_000) + 1;
let fee = 100_000_000u128; // 10%
let charged = gross.wrapping_mul(fee) / 1_000_000_000;
// The mathematically correct fee is gross / 10, but the u128
// multiplication can overflow before the division.
assert!(charged < gross / 10);
let amount = u128::MAX;
let saturated_fee = amount.saturating_mul(fee) / 1_000_000_000;
// Saturation prevents a panic, but it also computes a value from
// u128::MAX instead of the true amount * fee product.
assert!(saturated_fee < amount / 10);
```

Impact

Deposit fees, haircut, insurance fees, withdraw fees, or yield fees can be undercharged, and vault accounting can become inconsistent.

Recommendation

Use shared checked arithmetic helpers for percentage math and vault accounting. Compute `gross * fee / FEES_CONSTANT` with `U256`, then explicitly revert if the final result cannot fit in `u128`. Replace saturating yield-fee math with checked `mulDiv`, use checked add/sub for vault fields, and add boundary tests around `u128::MAX / fee` and vault counter limits.

Status

Acknowledged

[Q-02] Core - Missing defense-in-depth invariant checks

Description

Core authorizes `put()` and `exit()` by checking that `caller` is the registered asset issuer, but it does not independently enforce key asset, metadata, and position invariants before supply or deposit-accounting changes in `contracts/core/src/lib.rs:132-225`.

Vulnerability Details

Core relies on the registered issuer as the canonical validation boundary. The current canonical issuer contracts perform several of these checks before calling Core, so ordinary users cannot bypass them through intended entrypoints. If the registered issuer is malicious, misconfigured, or compromised, however, Core does not re-enforce two important invariant groups.

1. Asset status is not enforced in Core. The registry can disable or emergency-stop an asset, and the active state is `status == Enabled` in `libraries/stbl-core-types/src/asset_definition.rs:251-255`. `Core.put()` and `Core.exit()` fetch the asset and check issuer identity, but they do not enforce the intended active-state policy before minting, burning, or changing deposit accounting in `contracts/core/src/lib.rs:146-170` and `contracts/core/src/lib.rs:203-225`.
2. Position accounting data is not independently validated in Core. `Core.put()` accepts issuer-supplied `YLDMetadata` and mints USST from `meta.stable_value_net` in `contracts/core/src/lib.rs:152-177`. It does not validate that the metadata was derived from the deposited asset amount, configured fees, oracle price, or current registry configuration. The canonical issuer computes those fields before calling Core in `contracts/asset-issuer/src/lib.rs:424-468`, but Core does not enforce the same invariant at the mint boundary. The same trust issue exists on exit: `Core.exit()` accepts `asset_id`, `token_id`, and `value`, then burns USST/YLD and decrements deposits by the supplied value in `contracts/core/src/lib.rs:191-225`, without loading YLD metadata to require `metadata.asset_id == asset_id` or `metadata.stable_value_net == value`.

Proof of Concept

The invariant gaps can be demonstrated independently in a test harness:

1. Disable an asset, then call `Core.put()` as issuer; Core checks issuer identity but not active state in `contracts/core/src/lib.rs:146-170`.
2. Have the registered issuer call `Core.put()` with inflated `meta.stable_value_net`, or call `Core.exit()` with mismatched `asset_id`, `token_id`, or `value`; Core reaches mint, burn, or deposit-accounting logic using the supplied values.

Impact

The missing checks can corrupt accounting or mint unbacked USST.

Recommendation

Add Core-level invariant checks before supply or deposit-accounting changes:

1. `Core.put()` and `Core.exit()` should enforce the intended asset-status policy at the Core boundary.
2. `Core.put()` should validate issuer metadata against registry terms, oracle pricing, and fee configuration, or accept only a structured issuer attestation that Core can verify. `Core.exit()` should load metadata for `token_id`, require `metadata.asset_id == asset_id`, and derive the burn/decrement amount from metadata or require `metadata.stable_value_net == value`.

Status

Acknowledged

[Q-03] AssetIssuer#withdraw_fees - Token transfer precedes fee state update

Description

`withdraw_fees()` transfers fee tokens before clearing fee counters. The standard issuer calls `token transfer()` in `contracts/asset-issuer/src/lib.rs:561-570` and only zeroes fee buckets afterward in `contracts/asset-issuer/src/lib.rs:581-587`. The airdrop issuer repeats this in `contracts/airdrop-asset-issuer/src/lib.rs:621-647`.

Vulnerability Details

This is a checks-effects-interactions hardening issue. During the external token transfer, old nonzero fee buckets remain stored, so a nested `withdraw_fees()` call would see the same amount.

Exploitability is constrained by Soroban authorization: the nested call must satisfy `caller.require_auth()` and `caller == treasury` in `contracts/asset-issuer/src/lib.rs:539-548` or `contracts/airdrop-asset-issuer/src/lib.rs:599-608`. The risk appears only with broad treasury authorization, a permissive treasury contract wallet, or an auth tree permitting reentrant `withdraw_fees(treasury)`.

Proof of Concept

A malicious asset token can call back before `transfer()` returns:

```
// During token.transfer(from = issuer, to = treasury, amount):  
issuer.withdraw_fees(treasury);
```

If treasury authorization permits it, the second call runs before the outer call clears the fees.

Impact

Stale fee accounting can be reused.

Recommendation

Operationally, treasury signing should authorize only the exact intended `withdraw_fees` call. Avoid wildcard contract-wallet policies or broad reusable auth trees that allow



nested `withdraw_fees(treasury)`.

At the code level, follow CEI: snapshot fee values locally, zero and persist fee buckets, then call `token_client.transfer()`. Emit `FeesWithdrawnEvent` from the snapshot.

Status

Resolved



[Q-04] Upgrade safety checks are not mandatory across all WASM upgrade paths

Description

`ContractUpgrader` adds name and version checks around WASM upgrades, but those checks are not mandatory across all upgrade paths. Authorized upgraders can still use target-level entrypoints, and many contracts do not expose the wrapper's version interface.

Vulnerability Details

`upgrade_contract()` checks local `UPGRADER_ROLE`, reads `get_contract_version()`, calls target `upgrade()`, and then compares the post-upgrade name and version in `contracts/contracts-upgrader/src/lib.rs:26-46`. This policy applies only when operators use the wrapper.

Targets still expose direct upgrade entrypoints. The registry, core, and asset issuer update WASM after their own role checks in `contracts/register/src/lib.rs:171-176`, `contracts/core/src/lib.rs:111-119`, and `contracts/asset-issuer/src/lib.rs:127-141`. These paths skip the wrapper checks.

Coverage is partial because the wrapper uses `StblRegisterTrait`, whose version view is `get_contract_version()` in `libraries/stbl-core-interfaces/src/register.rs:7-73`. Many upgradeable contracts expose only `version()`. The wrapper also reads the new version after installing the new WASM, so it relies on self-reported metadata.

Proof of Concept

No unprivileged exploit proof is provided because every path requires authorized upgrade power. An `UPGRADER_ROLE` holder can call target `upgrade()` directly or deploy WASM that reports valid metadata.

Impact

Arbitrary WASM can replace protocol logic.

Recommendation

Enforce one upgrade policy. Targets should accept upgrades only from the approved upgrader contract, or implement hash allowlists and validation locally. Use multisig custody, timelocks, runbooks, pre-approved WASM hashes, and one version interface.

Status

Acknowledged



[Q-05] AssetOracle - Future oracle timestamps are not rejected

Description

AssetOracle checks whether an external oracle timestamp is too old, but it does not reject timestamps greater than the current ledger timestamp in `contracts/asset-oracle/src/lib.rs:240-250`.

Vulnerability Details

The stale-price check uses `timestamp + threshold <= now`. If an oracle adapter returns a future timestamp, the check treats the value as fresh until ledger time catches up. The accepted price is then cached and used by forward and inverse conversions in `contracts/asset-oracle/src/lib.rs:257-321`.

This is primarily an oracle-adapter hardening gap. A reputable oracle that stamps updates from `env.ledger().timestamp()` should not naturally produce future timestamps. The concern is limited to faulty adapters, unit mismatches, clock/configuration mistakes, or an already compromised oracle path.

Proof of Concept

A minimal test oracle that returns `(price, now + offset)` would show the behavior. After advancing ledger time beyond the normal threshold but before the future timestamp, `fetch_price()` would still succeed because `timestamp + threshold <= now` remains false in `contracts/asset-oracle/src/lib.rs:246-254`.

Impact

Future timestamps can keep an oracle value acceptable longer than intended.

Recommendation

Reject future timestamps before applying the stale-price threshold. A complete defensive check should require `timestamp <= now`, then calculate freshness with checked arithmetic such as `now - timestamp < threshold`. Add boundary tests for future timestamps, exact-threshold expiry, and fresh same-ledger timestamps.

Status

Resolved



[Q-06] AssetYieldDistributor#distribute_reward - Zero-stake rewards can remain unallocated

Description

`distribute_reward()` pulls reward tokens from the issuer before checking whether any YLD stake exists. If `current_total_supply == 0`, it updates the distribution timestamp and returns without increasing `reward_index`, leaving the transferred reward outside the normal claim accounting in `contracts/asset-yield-distributor/src/lib.rs:290-303`.

Vulnerability Details

Reward claims are based on each token's stake and the global reward index delta in `contracts/asset-yield-distributor/src/lib.rs:242-265`. When total supply is zero, `distribute_reward()` skips the reward-index update after already transferring tokens into the distributor. The airdrop issuer is the clearest path because external reward tokens can be sent to the issuer before any YLD position exists, then `distribute_yield()` approves and forwards that balance to the distributor in `contracts/airdrop-asset-issuer/src/lib.rs:538-584`.

Proof of Concept

This PoC proves that reward tokens can be transferred into the distributor while `total_supply` is zero, without increasing `reward_index`. Insert the test below into `integration-tests/tests/test_dpt_calls.rs`; the file's existing imports are sufficient.

```
#[test]
fn test_zero_stake_airdrop_distribution_strands_rewards() {
    let system = TestIntegratedSystem::deploy();
    let env = &system.env;
    let register_client = register::StblRegisterClient::new(env, &system.registry);
    let benji_decimals: u32 = 6;
    let benji_token = env.register(
        usst_token::UsstToken,
        (system.admin.clone(), Some(benji_decimals)),
    );
    let benji_token_client = StblTokenClient::new(env, &benji_token);
    system.grant_minting_access(&benji_token, &system.admin);
}
```

```

let asset_name = String::from_str(env, "BenJi");
let description = String::from_str(env, "Airdrop-backed DPT asset");
let asset_id =
    register_client.add_asset(&system.admin, &asset_name, &description, &1, &3,
&true);
let rwa_oracle = env.register(rwa_oracle::RwaOracle, (system.admin.clone(),));
let rwa_oracle_client = ExternalOracleClient::new(env, &rwa_oracle);
rwa_oracle_client.set_price_data(&1_000_000_000);
let oracle = env.register(
    asset_oracle::AssetOracle,
    (system.admin.clone(), rwa_oracle.clone(), &9u32, &2592000u64),
);
let issuer = env.register(
    AssetIssuer,
    (system.admin.clone(), system.registry.clone(), asset_id),
);
let issuer_access_control_client = AccessControlClient::new(env, &issuer);
issuer_access_control_client.grant_role(
    &system.admin,
    &system.treasury,
    &String::from_str(env, YD_ROLE),
);
let distributor = env.register(
    asset_yield_distributor::AssetYieldDistributor,
    (system.admin.clone(), system.registry.clone(), asset_id),
);
let fees: Vec<u128> = vec![env, 10, 10, 10, 10];
register_client.setup_asset(
    &system.admin,
    &asset_id,
    &vec![
        env,
        benji_token.clone(),
        issuer.clone(),
        distributor.clone(),
        oracle.clone(),
    ],
    &10,

```

```

        &u128::pow(10, 24),
        &fees,
        &10800,
        &3600,
        &Bytes::new(env),
    );
    let distributor_client = YieldDistributorClient::new(env, &distributor);
    let asset_vault_client = VaultClient::new(env, &issuer);
    assert_eq!(distributor_client.fetch_total_supply(), 0);
    assert_eq!(distributor_client.fetch_reward_index(), 0);
    let reward_amount: i128 = (100u128 * u128::pow(10, benji_decimals)) as i128;
    system.mint_tokens_to_user(&benji_token, &system.admin, Some(reward_amount as
u128));
    benji_token_client.transfer(&system.admin, &issuer, &reward_amount);
    let distributor_balance_before = benji_token_client.balance(&distributor);
    env.ledger().set(LedgerInfo {
        sequence_number: env.ledger().sequence() + (3605 / 5) as u32,
        timestamp: env.ledger().timestamp() + 3605,
        ..env.ledger().get()
    });
    asset_vault_client.distribute_yield(&system.treasury);
    let distributor_balance_after = benji_token_client.balance(&distributor);
    assert!(distributor_balance_after > distributor_balance_before);
    assert_eq!(distributor_client.fetch_total_supply(), 0);
    assert_eq!(distributor_client.fetch_reward_index(), 0);
}

```

Impact

The code can strand unallocated reward tokens in the distributor.

Recommendation

Check `current_total_supply` before transferring reward tokens. If no stake exists, revert or leave rewards pending in the issuer until a later distribution can allocate them. If zero-stake rewards should belong to the protocol, route them explicitly to treasury and emit a distinct event instead of treating them as a normal distribution.

Status

Acknowledged



[Q-07] YldSplitter - Minimum boundary differs from deposits

Description

Deposits and YLD splits apply different boundary rules for `min_deposit_amount`. Deposits reject `asset_value <= min_deposit_amount` in `contracts/asset-issuer/src/lib.rs:308-310`, and the airdrop issuer follows the same rule in `contracts/airdrop-asset-issuer/src/lib.rs:331-334`.

Vulnerability Details

The splitter rejects only split outputs strictly below the minimum. Split-by-count checks both the repeated part and remainder with `< min_deposit_amount` in `contracts/yld-splitter/src/lib.rs:97-123`, and split-by-ratio applies the same rule in `contracts/yld-splitter/src/lib.rs:171-174`.

An exact-minimum position cannot be created through deposit, but can be produced by splitting a larger YLD. This is an invariant mismatch rather than a demonstrated funds-loss path: the variable name can reasonably imply equality should be allowed, while deposits currently require values above the minimum.

Proof of Concept

With `min_deposit_amount = 1_000`, a direct exact-minimum deposit reverts, while splitting a `2_000` asset-value YLD into two equal parts succeeds.

```
assert!(asset_issuer.try_deposit(&alice, &1_000, &None).is_err());
yld_splitter.split_yld(&alice, &asset_id, &token_id, &2, &0, &true);
```

Impact

The protocol can create exact-minimum YLD positions through split flows even though deposit flows reject the same boundary. This may confuse integrations or operators that assume all creation paths enforce one minimum rule.

Recommendation

Choose and document one invariant. If exact-minimum positions are valid, make deposits reject only values below `min_deposit_amount`. If positions must be above the

minimum, make splitter checks reject `<= min_deposit_amount`. Add boundary tests for deposits and both split modes.

Status

Resolved



[Q-08] Privileged roles - Operational control can disrupt protocol

Description

The protocol relies on broad privileged roles for administration. This is an operational security risk rather than a code-level bypass, because the actions are gated by explicit role checks.

Vulnerability Details

`DEFAULT_ADMIN_ROLE` can update the core and treasury addresses in `contracts/register/src/lib.rs:185-198`, while `REGISTER_ROLE` can configure asset contracts, oracles, fees, limits, durations, and asset state in `contracts/register/src/lib.rs:272-349` and `contracts/register/src/lib.rs:489-566`. In particular, `setup_asset()` only rejects component addresses that equal the registry itself, so a privileged operator can wire arbitrary external token, issuer, reward distributor, and oracle contracts into an enabled asset in `contracts/register/src/lib.rs:317-338`. Downstream flows then trust the configured issuer and oracle when minting, redeeming, and pricing positions in `contracts/core/src/lib.rs:146-170`. `UPGRADER_ROLE` can replace contract Wasm through the upgrader and target upgrade paths in `contracts/contracts-upgrader/src/lib.rs:19-38` and `contracts/register/src/lib.rs:170-180`. `LIST_MANAGER_ROLE` can blacklist any USST account, and USST blacklist checks block mint recipients, bridge flows, transfers, and `transfer_from` in `contracts/usst-token/src/lib.rs:133-183` and `contracts/usst-token/src/lib.rs:208-223`.

Proof of Concept

No exploit proof is provided because these are intended permissioned entrypoints. The risk materializes operationally if role keys are compromised, mismanaged, or held by a single party without governance controls.

Impact

Critical protocol addresses can be redirected, implementations can be replaced, assets can be disabled, untrusted asset components can be wired in, or USST flows can be frozen.

Recommendation

Treat these roles as critical operational controls. Use multisig custody, role separation, monitoring, deployment runbooks, and timelocks or allowlists if the intended governance model requires constrained administration. For asset onboarding, validate configured component contracts with allowlists, expected asset ids, role checks, and interface or version fingerprints before enabling an asset.

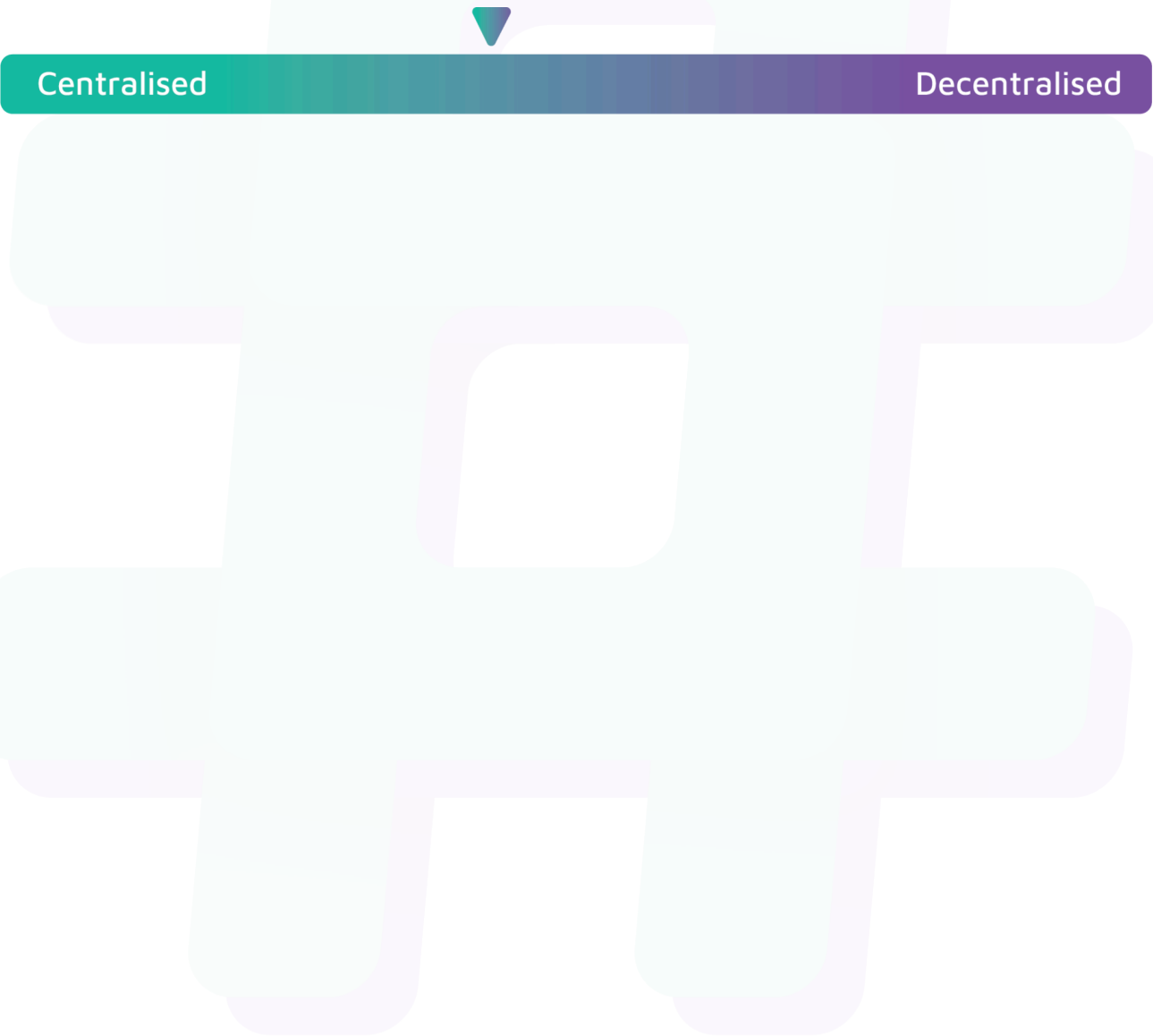
Status

Acknowledged

Centralisation

The STBL project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the STBL project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.